THE ULTIMATE PATTERN-BASED CODING INTERVIEW GUIDE

75 DSA Questions That Make DSA Easy

A curated, pattern-first path through the 75 problems that unlock every major coding-interview archetype — built for beginners, students, and engineers preparing for product-based companies.

75	14	30	Py
Curated Problems	Core Patterns	Day Plan	Python Solutions

FOR BEGINNERS · STUDENTS · FRESHERS · SWE INTERVIEW CANDIDATES

Curated & Compiled by Venu Uday

BEFORE YOU BEGIN

How to Use This Guide

This guide is not 75 random problems — it is a deliberately ordered curriculum. Every question was chosen because it teaches a reusable *pattern* that reappears across dozens of real interview questions. Master the pattern, and problems you've never seen start to feel familiar.

1. Read the pattern first

Before solving, read "Why This Question Matters" and "Pattern Used." Interviews reward pattern recognition, not memorized code.

2. Attempt before reading the solution

Spend 15–25 minutes attempting brute force, then optimal, before reading the walkthrough. Struggle is where learning happens.

3. Dry-run on paper

Trace the example by hand before running code. This is exactly what you'll do at a whiteboard or on a call.

4. Revisit, don't just move on

Use the Progress Tracker and the 30-Day Plan to schedule spaced revision, not a single pass through all 75.

PATTERN UNLOCKED 🔓

Every question in this book carries a **Pattern Used** label. When you meet a NEW problem in a real interview, your first job is to match it to one of the 14 patterns in the next section — that match is 80% of the battle.

SECTION 3

DSA Pattern Cheat Sheet

14 patterns. Learn to recognize the signal — the rest is execution.

🔓 Two Pointers

Recognize it when: Array or string is sorted (or can be), and you're comparing/combining elements from both ends or two speeds.

Signal phrases: “Sorted array” + “pair/triplet that sums to X” or “palindrome check”.

🔓 Sliding Window

Recognize it when: You need a contiguous subarray/substring satisfying a condition, and shrinking/growing a window is cheaper than recomputation.

Signal phrases: “Longest/shortest/max subarray or substring with condition Y”.

🔓 Prefix Sum

Recognize it when: You need repeated range-sum queries or a running total to detect a target difference.

Signal phrases: “Subarray sums to K” or “range sum queries”.

🔓 Fast & Slow Pointers

Recognize it when: You're traversing a linked list and need the middle, a cycle, or a cycle's start — without extra memory.

Signal phrases: “Linked list” + “cycle”, “middle”, or “nth from end”.

🔓 Binary Search on Answer

Recognize it when: The answer lies in a monotonic search space (if X works, everything above/below X also works).

Signal phrases: “Minimize the maximum” or “find the smallest value such that condition holds”.

🔓 Monotonic Stack

Recognize it when: You need the next/previous greater or smaller element for every position in one pass.

Signal phrases: “Next greater element”, “daily temperatures”, “largest rectangle”.

🔓 BFS

Recognize it when: You need the shortest path in an unweighted graph/grid, or level-by-level processing.

Signal phrases: “Shortest path”, “minimum steps”, “level order”, unweighted edges.

🔓 DFS

Recognize it when: You need to explore every path, detect connectivity, or recurse into a tree/graph structure.

Signal phrases: “Number of islands”, “connected components”, tree traversal.

🔓 Topological Sort

Recognize it when: You have a directed graph with dependency ordering (prerequisites) and need a valid order or cycle check.

Signal phrases: “Course schedule”, “build order”, “task scheduling with prerequisites”.

🔓 Heap / Priority Queue

Recognize it when: You repeatedly need the current min or max (Kth largest, merge sorted streams, running median).

Signal phrases: “Kth largest/smallest”, “top K”, “merge K sorted lists”.

🔓 Backtracking

Recognize it when: You must generate all valid combinations/permutations/placements and can prune invalid branches early.

Signal phrases: “All subsets/permutations”, “place N queens”, “word search on a grid”.

🔓 Greedy

Recognize it when: A locally optimal choice at each step provably leads to the global optimum — no need to explore alternatives.

Signal phrases: “Minimum number of intervals/jumps”, “maximum non-overlapping”.

🔓 1D / 2D DP

Recognize it when: The problem has overlapping subproblems and optimal substructure — the brute force recomputes the same state repeatedly.

Signal phrases: “Count ways to...”, “minimum/maximum cost to...”, “longest/shortest sequence”.

SECTION 4

Big-O Complexity Quick Reference

Complexity	Name	Example Operation
`O(1)`	Constant	Hash map lookup, array index access
`O(log n)`	Logarithmic	Binary search, balanced BST operations
`O(n)`	Linear	Single pass over an array/list
`O(n log n)`	Linearithmic	Efficient sorting (merge/quick/heap sort)
`O(n\u00b2)`	Quadratic	Nested loops, naive pair comparisons
`O(2^n)`	Exponential	Unmemoized recursion, subsets/power sets
`O(n!)`	Factorial	Generating all permutations

Common Data Structure Costs

Array access O(1) · Array search O(n) · Hash map insert/lookup O(1) avg · BST search/insert O(log n) avg · Heap push/pop O(log n) · Heap peek O(1)

Rule of Thumb for n

n \u2264 20: O(2^n)/O(n!) OK · n \u2264 1,000: O(n\u00b2)/O(n\u00b3) OK · n \u2264 10^6: O(n log n) needed · n \u2264 10^9: O(log n)/O(1) needed

SECTION 5

30-Day Completion Plan

Days	Focus	Questions
1–2	Arrays & Hashing	Q1–Q7
3–4	Two Pointers	Q8–Q12
5–6	Sliding Window	Q13–Q17
7–8	Binary Search	Q18–Q23
9–10	Strings	Q24–Q28
11–12	Linked Lists	Q29–Q34
13–14	Stack & Queue	Q35–Q39
15–17	Trees & BST	Q40–Q46
18–19	Heap / Priority Queue	Q47–Q50
20–21	Backtracking	Q51–Q55
22–24	Graphs	Q56–Q62
25–27	Dynamic Programming	Q63–Q69
28	Greedy	Q70–Q73
29	Bit Manipulation	Q74–Q75
30	Full Revision Day	Top 15 (see Section 7) + weak patterns

SECTION 6

75 Curated DSA Problems

Ordered beginner → intermediate → interview level. Each problem includes a full walkthrough and a working Python solution.

CATEGORY 1 OF 14

Arrays & Hashing

The foundation pattern set — hash maps turn O(n²) brute force into O(n). Master these first.

7 questions in this section

QUESTION 1 OF 75

Easy

Two Sum

Pattern: Hashing (index lookup)

WHY THIS QUESTION MATTERS

The single most common interview opener — teaches you that a hash map turns 'have I seen X' into O(1).

Problem Statement

Given an array of integers nums and an integer target, return the indices of the two numbers that add up to target. Each input has exactly one solution, and you may not use the same element twice.

Example

```
Input: nums = [2,7,11,15], target = 9
Output: [0,1]  (because
nums[0]+nums[1] == 9)
```

Constraints

2 ≤ nums.length ≤ 10^4 · -10^9 ≤ nums[i] ≤ 10^9 · exactly one valid answer exists

Brute-Force Approach

Try every pair with two nested loops, checking if they sum to target. O(n²) time, O(1) space.

Optimal Approach

Walk the array once. For each number, check if (target - number) is already in a hash map of value→index. If yes, you found the pair; otherwise store the current number and its index.

Step-by-Step Intuition

Instead of asking 'what pairs sum to target' after the fact, ask 'what number would I still need' at each step, and remember every number you've already seen so the lookup is instant.

Dry Run

`nums=[2,7,11,15], target=9. i=0, num=2, need=7, map={} → not found, store {2:0}. i=1, num=7, need=2, map={2:0} → found! return [0,1].`

Time Complexity

`O(n) — single pass, O(1) hash map operations.`

Space Complexity

`O(n) — hash map can hold up to n entries.`

Common Mistakes

Using the same element twice; forgetting to check 'need' before inserting the current number (which would let an element pair with itself incorrectly); assuming the array is sorted.

Interview Follow-up Questions

• What if the array is sorted — can you do O(1) space?

• What if there are multiple valid pairs and you need all of them?

• How would you handle streaming input where nums arrive one at a time?

Clean Python Solution

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        need = target - num
        if need in seen:
            return [seen[need], i]
        seen[num] = i
    return []
```

KEY TAKEAWAY

When you need to find a complement of the current element, a hash map converts an O(n²) search into O(n).

QUESTION 2 OF 75

Easy

Contains Duplicate

Pattern: Hashing (set membership)

WHY THIS QUESTION MATTERS

Builds the reflex of reaching for a set whenever the question is 'has this been seen before'.

Problem Statement

Given an integer array nums, return true if any value appears at least twice, and false if every element is distinct.

Example

```
Input: nums = [1,2,3,1] → Output: true
Input: nums = [1,2,3,4] → Output:
false
```

Constraints

1 ≤ nums.length ≤ 10^5 · -10^9 ≤ nums[i] ≤ 10^9

Brute-Force Approach

Compare every pair of elements, O(n²); or sort then scan adjacent pairs, O(n log n).

Optimal Approach

Add each number to a hash set; if a number is already in the set, return true immediately.

Step-by-Step Intuition

A set gives O(1) membership checks, so you can detect a repeat the moment it happens instead of comparing every pair.

Dry Run

`nums=[1,2,3,1]. seen={}. 1 not in seen → add. 2 not in seen → add. 3 not in seen → add. 1 IS in seen → return True.`

Time Complexity

`O(n) average.`

Space Complexity

`O(n) for the set.`

Common Mistakes

Sorting first when O(n) is achievable and expected; forgetting early-exit (checking membership before insertion, not after).

Interview Follow-up Questions

• What if memory is extremely constrained — can sorting in place beat the hash set on space?

• How would you find the actual duplicate value, not just true/false?

Clean Python Solution

```
def contains_duplicate(nums):
    seen = set()
    for num in nums:
        if num in seen:
            return True
        seen.add(num)
    return False
```

KEY TAKEAWAY

'Have I seen this before' almost always means: reach for a hash set.

QUESTION 3 OF 75

Easy

Valid Anagram

Pattern: Hashing (frequency count)

WHY THIS QUESTION MATTERS

Introduces frequency-counting, the base of many string/array comparison problems.

Problem Statement

Given two strings s and t, return true if t is an anagram of s (same characters, same counts, possibly different order).

Example

`Input: s = "anagram", t = "nagaram" → Output: true`

`Input: s = "rat", t = "car" → Output: false`

Constraints

1 ≤ s.length, t.length ≤ 5×10^4 · lowercase English letters

Brute-Force Approach

Sort both strings and compare, O(n log n).

Optimal Approach

Count character frequencies in s with a hash map/array, then decrement while scanning t; if any count goes negative or lengths differ, return false.

Step-by-Step Intuition

An anagram means identical multisets of characters. A frequency table is a multiset with O(1) lookups.

> **Dry Run**
> `s="rat" → counts {r:1,a:1,t:1}. Scan t="car": c not in counts → return False immediately.`

Time Complexity
`O(n) where n = length of the strings.`

Space Complexity
`O(1) if the alphabet size is fixed (26 letters), else O(k) for k distinct characters.`

Common Mistakes

Not checking lengths first as a fast fail; using sorted() when interviewer wants O(n); ignoring Unicode/uppercase edge cases.

Interview Follow-up Questions

- What if the inputs contain Unicode characters, not just lowercase letters?
- How would you check if s is an anagram of t while ignoring spaces and case?

Clean Python Solution

```
from collections import Counter
def is_anagram(s, t):
    if len(s) != len(t):
        return False
    return Counter(s) == Counter(t)
```

KEY TAKEAWAY

Frequency maps let you compare 'same multiset of items' in linear time.

QUESTION 4 OF 75

Medium

Group Anagrams

Pattern: Hashing (canonical key)

WHY THIS QUESTION MATTERS

Teaches the 'canonical key' trick — map many different-looking inputs to the same bucket.

Problem Statement

Given an array of strings strs, group the anagrams together. Return the answer in any order.

Example

Constraints

1 ≤ strs.length ≤ 10^4 · 0 ≤ strs[i].length ≤ 100 · lowercase letters

```
Input: strs =
["eat","tea","tan","ate","nat","bat"]
Output: [["bat"],["nat","tan"],
["ate","eat","tea"]]
```

Brute-Force Approach

Compare every string to every other string for the anagram property, O(n² · k log k).

Optimal Approach

For each string, compute a canonical key (sorted string, or a 26-length count tuple) and group strings sharing a key in a hash map.

Step-by-Step Intuition

Anagrams share the same sorted form or the same letter-count signature — use that signature as a hash map key so grouping becomes O(1) per insert.

> **Dry Run**
> `"eat"→key "aet". "tea"→"aet" (same bucket). "tan"→"ant" (new bucket). Continue until all strings are bucketed by key.`

Time Complexity
`O(n · k log k) with sorting keys (n strings, k = max length), or O(n · k) using count-tuples.`

Space Complexity
`O(n · k) to store all strings in the map.`

Common Mistakes

Using sorted string as key when k is large (count-tuple is faster); forgetting strings can be empty; mutating the input list.

Interview Follow-up Questions

• Can you build the key in O(k) instead of O(k log k)?

• How would this scale if strs had 10^7 entries?

Clean Python Solution

```python
from collections import defaultdict
def group_anagrams(strs):
    groups = defaultdict(list)
    for s in strs:
        key = tuple(sorted(s))
        groups[key].append(s)
    return list(groups.values())
```

KEY TAKEAWAY

When grouping by a shared property, hash on a canonical signature of that property.

QUESTION 5 OF 75

Medium

Top K Frequent Elements

Pattern: Hashing + Bucket Sort

WHY THIS QUESTION MATTERS

Shows that sorting isn't always optimal — bucket sort exploits bounded frequency to reach O(n).

Problem Statement

Given an integer array nums and integer k, return the k most frequent elements.

Example

```
Input: nums = [1,1,1,2,2,3], k = 2 →
Output: [1,2]
```

Constraints

1 ≤ nums.length ≤ 10^5 · k is always valid (1 ≤ k ≤ number of distinct elements)

Brute-Force Approach

Count frequencies, sort by frequency descending, take top k. O(n log n).

Optimal Approach

Count frequencies with a hash map, then bucket elements by frequency (bucket index = frequency, capped at n). Walk buckets from high to low frequency collecting k elements.

Step-by-Step Intuition

Frequency can never exceed n, so instead of sorting arbitrary values you can index directly into 'frequency buckets' — an O(n) counting sort.

Dry Run

```
nums=[1,1,1,2,2,3]. counts={1:3,2:2,3:1}. buckets[3]=[1], buckets[2]=[2],
buckets[1]=[3]. Walk from bucket 6 down: bucket[3]=[1] → take 1, bucket[2]=[2]
→ take 2. k=2 reached → [1,2].
```

Time Complexity

```
O(n) using bucket sort (vs O(n log
n) with a heap/sort).
```

Space Complexity

```
O(n) for counts and buckets.
```

Common Mistakes

Using a full sort when O(n) bucket sort is expected in a follow-up; off-by-one on bucket size (max frequency = n).

Interview Follow-up Questions

• Could you solve this with a min-heap of size k instead? What's the time complexity?

• What if k could equal n (return all elements sorted by frequency)?

Clean Python Solution

```
from collections import Counter
def top_k_frequent(nums, k):
    count = Counter(nums)
    buckets = [[] for _ in range(len(nums) + 1)]
    for num, freq in count.items():
        buckets[freq].append(num)
    result = []
    for freq in range(len(buckets) - 1, 0, -1):
        for num in buckets[freq]:
            result.append(num)
            if len(result) == k:
                return result
    return result
```

KEY TAKEAWAY

When a value is bounded by n (like frequency), bucket sort beats comparison sort.

QUESTION 6 OF 75

Medium

Product of Array Except Self

Pattern: Prefix / Suffix Products

WHY THIS QUESTION MATTERS

Classic no-division constraint problem — teaches prefix/suffix accumulation, a building block for prefix sum too.

Problem Statement

Given an integer array nums, return an array answer where answer[i] is the product of all elements except nums[i], without using division, in O(n) time.

Example

`Input: nums = [1,2,3,4] → Output: [24,12,8,6]`

Constraints

2 ≤ nums.length ≤ 10^5 · product of any prefix/suffix fits in a 32-bit integer

Brute-Force Approach

For each i, multiply all other elements: O(n²).

Optimal Approach

Build a prefix-product array (product of everything to the left of i) and a suffix-product array (product of everything to the right), then answer[i] = prefix[i] * suffix[i]. Collapse suffix into a running variable to hit O(1) extra space.

Step-by-Step Intuition

'Everything except index i' = 'everything before i' × 'everything after i'. Precompute both directions once instead of recomputing per index.

> **Dry Run**

nums=[1,2,3,4]. prefix=[1,1,2,6]. suffix (right to left running product)=24,12,4,1 → combine: answer[0]=1*24=24, answer[1]=1*12=12, answer[2]=2*4=8, answer[3]=6*1=6.

Time Complexity
O(n) — two linear passes.

Space Complexity
O(1) extra (excluding the output array) using the running-suffix trick.

Common Mistakes

Reaching for division then special-casing zeros (fragile); allocating a separate suffix array when the running-variable trick avoids it.

Interview Follow-up Questions

• How do you handle the array containing one or more zeros?

• Can you do this with O(1) extra space, not counting the output?

Clean Python Solution

```
def product_except_self(nums):
    n = len(nums)
    answer = [1] * n
    prefix = 1
    for i in range(n):
        answer[i] = prefix
        prefix *= nums[i]
    suffix = 1
    for i in range(n - 1, -1, -1):
        answer[i] *= suffix
        suffix *= nums[i]
    return answer
```

KEY TAKEAWAY

When you can't use an operation directly (division), precompute both directions and combine.

QUESTION 7 OF 75

Medium

Subarray Sum Equals K

Pattern: Prefix Sum + Hashing

WHY THIS QUESTION MATTERS

THE canonical Prefix Sum pattern question — unlocks an entire family of 'subarray with property X' problems.

Problem Statement

Given an array of integers nums and an integer k, return the total number of contiguous subarrays whose sum equals k.

Example

Constraints

Input: nums = [1,1,1], k = 2 → Output: 2 (subarrays [1,1] and [1,1])

1 ≤ nums.length ≤ 2×10^4 · -1000 ≤ nums[i] ≤ 1000 ≤ k ≤ 10^7

Brute-Force Approach

Check every subarray's sum with nested loops: O(n²).

Optimal Approach

Keep a running prefix sum and a hash map of {prefix_sum: count_seen}. At each index, if (running_sum - k) exists in the map, add its count to the answer — that means some earlier prefix, when removed, leaves exactly k.

Step-by-Step Intuition

sum(i..j) = prefix[j] - prefix[i-1]. So sum(i..j) == k is the same as prefix[i-1] == prefix[j] - k. Storing how many times each prefix sum has occurred turns this into an O(1) lookup per index.

> **Dry Run**
>
> nums=[1,1,1], k=2. running=0, map={0:1}. i=0: running=1, need=1-2=-1, not in map, map={0:1,1:1}. i=1: running=2, need=0, map[0]=1 → count+=1, map={0:1,1:1,2:1}. i=2: running=3, need=1, map[1]=1 → count+=1. Total=2.

Time Complexity

O(n) — one pass with O(1) hash map operations.

Space Complexity

O(n) for the prefix-sum map.

Common Mistakes

Forgetting to seed the map with {0:1} (accounts for a prefix itself summing to k); recomputing sums from scratch per subarray.

Interview Follow-up Questions

• What if you needed the actual subarrays, not just the count?

• How would you handle a streaming array where k can change?

Clean Python Solution

```
from collections import defaultdict
def subarray_sum(nums, k):
    count = defaultdict(int)
    count[0] = 1
    running = 0
    total = 0
    for num in nums:
        running += num
        total += count[running - k]
        count[running] += 1
    return total
```

KEY TAKEAWAY

Whenever you see 'contiguous subarray sums to X', think prefix sum + hash map, not nested loops.

CATEGORY 2 OF 14

Two Pointers

Two indices moving toward or away from each other collapse nested loops into a single linear pass.

5 questions in this section

QUESTION 8 OF 75

Easy

Valid Palindrome

Pattern: Two Pointers (converging)

WHY THIS QUESTION MATTERS

The simplest two-pointer template — pointers start at both ends and move inward.

Problem Statement

Given a string s, return true if it reads the same forward and backward after converting to lowercase and removing all non-alphanumeric characters.

Example

`Input: s = "A man, a plan, a canal: Panama" → Output: true`

Constraints

1 ≤ s.length ≤ 2×10^5 · s consists of printable ASCII characters

Brute-Force Approach

Build a cleaned, lowercased copy of the string, then compare it to its reverse: O(n) time, O(n) space.

Optimal Approach

Two pointers start at both ends; skip non-alphanumeric characters; compare lowercased characters; converge to the middle.

Step-by-Step Intuition

You don't need a separate cleaned string — walk inward from both sides, skipping junk characters on the fly, comparing as you go.

Dry Run

`s="A man, a plan, a canal: Panama". left=0('A'), right=len-1('a'). Compare 'a'=='a' → match, move both inward, skipping spaces/punctuation as encountered, until pointers cross — all comparisons match → True.`

Time Complexity

`O(n) — single pass, pointers only move inward.`

Space Complexity

`O(1) — no extra string built.`

Common Mistakes

Building a new string (works, but not O(1) space as follow-up expects); forgetting to lowercase before comparing; mishandling pointers skipping past each other.

Interview Follow-up Questions

• Can you solve it in O(1) space (in-place, no cleaned copy)?

• What if at most one character can be removed to make it a palindrome (Valid Palindrome II)?

Clean Python Solution

```
def is_palindrome(s):
    left, right = 0, len(s) - 1
    while left < right:
        while left < right and not s[left].isalnum():
            left += 1
        while left < right and not s[right].isalnum():
            right -= 1
        if s[left].lower() != s[right].lower():
            return False
        left += 1
        right -= 1
    return True
```

KEY TAKEAWAY

Converging two pointers avoid building an extra copy of the data whenever you're comparing from both ends.

QUESTION 9 OF 75

Medium

Two Sum II – Sorted Array

Pattern: Two Pointers (converging)

WHY THIS QUESTION MATTERS

Shows how sortedness turns a hash-map problem into an O(1)-space two-pointer problem.

Problem Statement

Given a 1-indexed array of integers numbers sorted in non-decreasing order, find two numbers that add up to target and return their indices (1-indexed).

Example

`Input: numbers = [2,7,11,15], target = 9 → Output: [1,2]`

Constraints

2 ≤ numbers.length ≤ 3×10^4 · numbers is sorted ascending · exactly one solution exists

Brute-Force Approach

For each i, binary search for target - numbers[i]: O(n log n). Or use a hash map (ignoring sortedness): O(n) time, O(n) space.

Optimal Approach

Two pointers at both ends. If the sum is too big, move the right pointer left; if too small, move the left pointer right; if equal, return.

Step-by-Step Intuition

Because the array is sorted, moving the low pointer only increases the sum and moving the high pointer only decreases it — so you can always tell which direction to move.

> **Dry Run**
> `numbers=[2,7,11,15], target=9. left=0(2), right=3(15), sum=17>9 → right--. right=2(11), sum=2+11=13>9 → right--. right=1(7), sum=2+7=9 → return [1,2] (1-indexed).`

Time Complexity
`O(n) — pointers move inward at most n times total.`

Space Complexity
`O(1) — no extra structures.`

Common Mistakes

Ignoring the sorted property and using the Two Sum hash-map solution (works, but wastes the O(1)-space opportunity); forgetting the 1-indexed return.

Interview Follow-up Questions

- What changes if the array can contain duplicates and you need all unique pairs?
- How would you adapt this to Three Sum?

Clean Python Solution

```
def two_sum_sorted(numbers, target):
    left, right = 0, len(numbers) - 1
    while left < right:
        total = numbers[left] + numbers[right]
        if total == target:
            return [left + 1, right + 1]
        elif total < target:
            left += 1
        else:
            right -= 1
    return []
```

KEY TAKEAWAY

A sorted array is a strong signal to try two pointers before reaching for extra memory.

QUESTION 10 OF 75

Medium

3Sum

Pattern: Two Pointers (fix + converge)

WHY THIS QUESTION MATTERS

Extends the two-pointer template to a fixed outer loop — the pattern behind 4Sum and k-Sum generally.

Problem Statement

Given an integer array nums, return all unique triplets [nums[i], nums[j], nums[k]] such that i != j != k and nums[i] + nums[j] + nums[k] == 0.

Example

```
Input: nums = [-1,0,1,2,-1,-4] →
Output: [[-1,-1,2],[-1,0,1]]
```

Constraints

3 ≤ nums.length ≤ 3000 · -10^5 ≤ nums[i] ≤ 10^5

Brute-Force Approach

Three nested loops checking every triplet: O(n³), plus a set to dedupe.

Optimal Approach

Sort the array. Fix the first element, then use two pointers on the remaining sorted subarray to find pairs summing to -nums[i], skipping duplicates at every level.

Step-by-Step Intuition

Sorting lets you fix one number and reduce the rest to a Two Sum II (converging two pointers) problem — and sortedness also makes duplicate-skipping trivial (just compare to the previous element).

Dry Run

```
Sorted: [-4,-1,-1,0,1,2]. i=0(-4): need pair summing to 4 from rest — none found. i=1(-1): need pair summing to 1: left=2(-1),right=5(2), sum=1 → triplet [-1,-1,2]; move both, skip dup -1's. left=3(0),right=4(1), sum=1 → triplet [-1,0,1].
```

Time Complexity

O(n²) — O(n log n) sort + O(n) two-pointer scan per fixed element.

Space Complexity

O(1) extra (excluding sort and output), or O(log n)/O(n) depending on sort implementation.

Common Mistakes

Forgetting to skip duplicate values at all three positions (produces duplicate triplets); not sorting first; off-by-one when advancing pointers past duplicates.

Interview Follow-up Questions

• How would you extend this to 4Sum?

• What if you needed triplets closest to a target instead of exactly zero (3Sum Closest)?

Clean Python Solution

```
def three_sum(nums):
    nums.sort()
    result = []
    n = len(nums)
    for i in range(n - 2):
        if i > 0 and nums[i] == nums[i - 1]:
            continue
        left, right = i + 1, n - 1
        while left < right:
            total = nums[i] + nums[left] + nums[right]
            if total < 0:
                left += 1
            elif total > 0:
                right -= 1
            else:
                result.append([nums[i], nums[left], nums[right]])
                left += 1
                right -= 1
                while left < right and nums[left] == nums[left - 1]:
                    left += 1
                while left < right and nums[right] == nums[right + 1]:
                    right -= 1
    return result
```

KEY TAKEAWAY

Sorting + fixing one element reduces k-Sum problems to a two-pointer subproblem.

QUESTION 11 OF 75

Medium

Container With Most Water

Pattern: Two Pointers (greedy converge)

WHY THIS QUESTION MATTERS

Teaches a greedy two-pointer proof: moving the shorter wall inward is the only move that can improve the answer.

Problem Statement

Given n non-negative integers height[i] representing vertical lines, find two lines that together with the x-axis form a container holding the most water. Return the max area.

Example

```
Input: height = [1,8,6,2,5,4,8,3,7] →
Output: 49
```

Constraints

2 ≤ height.length ≤ 10^5 · 0 ≤ height[i] ≤ 10^4

Brute-Force Approach

Try every pair of lines and compute area: O(n²).

Optimal Approach

Two pointers at both ends. Compute area = width × min(height[left], height[right]). Move the pointer at the shorter line inward (moving the taller one can never increase area).

Step-by-Step Intuition

Width only shrinks as pointers move inward, so area can only grow by increasing the limiting (shorter) height. Moving the taller line can't help — it's never the bottleneck — so always move the shorter one.

> **Dry Run**
> `height=[1,8,6,2,5,4,8,3,7]. left=0(1), right=8(7): area=8*1=8, move left (shorter). left=1(8), right=8(7): area=7*7=49, move right (shorter). Continue — best found is 49.`

Time Complexity
`O(n) — pointers move inward once each.`

Space Complexity
`O(1).`

Common Mistakes

Moving the taller pointer (breaks the greedy proof, misses the optimum); recomputing area with a nested loop.

Interview Follow-up Questions

• Can you prove why moving the shorter line is always safe?

• What if you needed the actual pair of indices, not just the area?

Clean Python Solution

```
def max_area(height):
    left, right = 0, len(height) - 1
    best = 0
    while left < right:
        area = (right - left) * min(height[left], height[right])
        best = max(best, area)
        if height[left] < height[right]:
            left += 1
        else:
            right -= 1
    return best
```

KEY TAKEAWAY

When width shrinks monotonically, moving the limiting (smaller) side is the only move that can improve the result.

QUESTION 12 OF 75

Hard

Trapping Rain Water

Pattern: Two Pointers (running max)

WHY THIS QUESTION MATTERS

A rite-of-passage hard problem that upgrades the two-pointer template with running maximums on both sides.

Problem Statement

Given n non-negative integers representing an elevation map, compute how much water it can trap after raining.

Example

`Input: height = [0,1,0,2,1,0,1,3,2,1,2,1] → Output: 6`

Constraints

1 ≤ height.length ≤ 2×10^4 · 0 ≤ height[i] ≤ 10^5

Brute-Force Approach

For each index, scan left and right to find the max wall on each side, water[i] = min(leftMax, rightMax) - height[i]. O(n²).

Optimal Approach

Precompute leftMax and rightMax arrays in O(n), or use two pointers with running leftMax/rightMax, moving the pointer on the side with the smaller running max (that side's water level is already determined).

Step-by-Step Intuition

Water trapped at index i is bounded by the SHORTER of the two tallest walls on either side. Whichever side currently has the smaller running max, its bound is already fixed — no wall further away on the other side can lower it — so you can safely resolve that side immediately.

Dry Run

`height=[0,1,0,2,1,0,1,3,2,1,2,1]. left=0,right=11, leftMax=0,rightMax=1. height[left]=0<=leftMax=0 initial... proceeding pointer by pointer accumulates water=6 total by the time pointers meet.`

Time Complexity

`O(n) with the two-pointer or prefix-array approach.`

Space Complexity

`O(1) with two pointers (vs O(n) for the prefix-array version).`

Common Mistakes

Only tracking one side's max (must track both leftMax and rightMax); off-by-one in when to add water vs update max.

Interview Follow-up Questions

- Can you also solve it with a monotonic stack?
- How would this generalize to a 2D elevation map (Trapping Rain Water II)?

Clean Python Solution

```
def trap(height):
    if not height:
        return 0
    left, right = 0, len(height) - 1
    left_max, right_max = height[left], height[right]
    water = 0
    while left < right:
        if left_max < right_max:
            left += 1
            left_max = max(left_max, height[left])
            water += left_max - height[left]
        else:
            right -= 1
            right_max = max(right_max, height[right])
            water += right_max - height[right]
    return water
```

KEY TAKEAWAY

When a value depends on the min of two running maximums, resolve the side with the smaller max first — it's already determined.

CATEGORY 3 OF 14

Sliding Window

A window that grows and shrinks over a sequence — the go-to pattern for subarray/substring problems.

5 questions in this section

QUESTION 13 OF 75

Easy

Best Time to Buy and Sell Stock

Pattern: Sliding Window (single pass min-track)

WHY THIS QUESTION MATTERS

The simplest sliding-window/single-pass problem — builds the habit of tracking a running minimum while scanning.

Problem Statement

Given an array prices where prices[i] is the stock price on day i, choose a single day to buy and a later day to sell to maximize profit. Return the max profit, or 0 if none is possible.

Example

```
Input: prices = [7,1,5,3,6,4] →
Output: 5  (buy at 1, sell at 6)
```

Constraints

1 ≤ prices.length ≤ 10^5 · 0 ≤ prices[i] ≤ 10^4

Brute-Force Approach

Try every buy-sell pair with nested loops: O(n²).

Optimal Approach

Scan once, tracking the minimum price seen so far and the best profit (current price - min so far) at each step.

Step-by-Step Intuition

The best sell day only matters relative to the lowest price seen BEFORE it — you never need to look backward more than once, so a running minimum captures all the history you need.

Dry Run

```
prices=[7,1,5,3,6,4]. minPrice=7,profit=0. price=1: minPrice=1. price=5:
profit=max(0,5-1)=4. price=3: profit stays 4. price=6: profit=max(4,6-1)=5.
price=4: profit stays 5. Answer=5.
```

Time Complexity

O(n) — one pass.

Space Complexity

O(1).

Common Mistakes

Nested-loop brute force when O(n) is expected; forgetting profit can be 0 (no valid transaction); updating minPrice after computing profit instead of using the pre-update value from earlier days only.

Interview Follow-up Questions

• What if you can make unlimited transactions (Best Time II)?

• What if there's a cooldown period after selling?

Clean Python Solution

```
def max_profit(prices):
    min_price = float('inf')
    profit = 0
    for price in prices:
        min_price = min(min_price, price)
        profit = max(profit, price - min_price)
    return profit
```

KEY TAKEAWAY

A single running minimum (or maximum) turns an O(n²) comparison problem into O(n).

QUESTION 14 OF 75

Medium

Longest Substring Without Repeating Characters

Pattern: Sliding Window (variable size)

WHY THIS QUESTION MATTERS

THE canonical variable-size sliding window — the template for dozens of substring problems.

Problem Statement

Given a string s, find the length of the longest substring without repeating characters.

Example

`Input: s = "abcabcbb" → Output: 3 ("abc")`

Constraints

0 ≤ s.length ≤ 5×10^4 · s consists of English letters, digits, symbols, and spaces

Brute-Force Approach

Check every substring for uniqueness: O(n³), or O(n²) with a set per starting index.

Optimal Approach

Expand a right pointer over the string; keep a hash set (or last-seen-index map) of characters in the current window; when a repeat is found, shrink from the left until the repeat is resolved. Track the max window size seen.

Step-by-Step Intuition

Instead of restarting the window at every repeat, jump the left pointer directly past the previous occurrence — the window only ever grows or shrinks, it never restarts from scratch.

> **Dry Run**

s="abcabcbb". window=[], left=0. add a,b,c → window "abc", len 3. next 'a' repeats — move left past first 'a': window becomes "bca"+'a'... using last-index map: left=index_of('a')+1=1. window "bcab"→dedupe to "bca", len stays 3. Continue: max length found = 3.

Time Complexity

O(n) — each character is visited at most twice (once by right, once by left).

Space Complexity

O(min(n, alphabet size)) for the window's character set/map.

Common Mistakes

Using a naive left+=1 loop instead of jumping left directly to last_seen[char]+1 (still correct but can be less efficient to reason about); forgetting the last-seen index might be outside the current window (must check left <= last_seen[char]).

Interview Follow-up Questions

• Can you solve this with at most K distinct characters allowed?

• What if you need the actual substring, not just its length?

Clean Python Solution

```
def length_of_longest_substring(s):
    last_seen = {}
    left = 0
    best = 0
    for right, ch in enumerate(s):
        if ch in last_seen and last_seen[ch] >= left:
            left = last_seen[ch] + 1
        last_seen[ch] = right
        best = max(best, right - left + 1)
    return best
```

KEY TAKEAWAY

A variable-size window with a map of 'last seen index' avoids restarting the scan on every repeat.

QUESTION 15 OF 75

Medium

Longest Repeating Character Replacement

Pattern: Sliding Window (count-based)

WHY THIS QUESTION MATTERS

Extends sliding window with a frequency count and a 'can I afford this window' invariant — a very common exam pattern.

Problem Statement

Given a string s and an integer k, you can replace up to k characters in the string with any other character. Return the length of the longest substring containing the same letter after such replacements.

Example

Constraints

Input: s = "ABAB", k = 2 → Output: 4

1 ≤ s.length ≤ 10^5 · s consists of uppercase English letters · 0 ≤ k ≤ s.length

Brute-Force Approach

Try every substring and check if (length - count of most frequent char) ≤ k: O(n²) or O(n³).

Optimal Approach

Sliding window with a character-count map. Track maxFreq (the most frequent character's count in the current window). If (windowSize - maxFreq) > k, the window is invalid — shrink from the left. Track the max valid window size.

Step-by-Step Intuition

A window is achievable if you only need to replace 'windowSize - maxFreq' characters — the ones that AREN'T the majority character. As long as that's ≤ k, the window is valid; the window never needs to shrink below its best-ever size, only slide.

> **Dry Run**
> s="ABAB", k=2. right=0 'A': counts{A:1}, maxFreq=1, size=1, 1-1=0≤2 valid. right=1 'B': counts{A:1,B:1}, maxFreq=1, size=2, 2-1=1≤2 valid. right=2 'A': counts{A:2,B:1}, maxFreq=2, size=3, 3-2=1≤2 valid. right=3 'B': counts{A:2,B:2}, maxFreq=2, size=4, 4-2=2≤2 valid. Answer=4.

Time Complexity
O(n) — 26 possible characters means maxFreq recomputation is bounded.

Space Complexity
O(1) — fixed 26-letter count array.

Common Mistakes

Shrinking the window even when it's still the best size so far (the window size only needs to be non-decreasing, not always minimal); recomputing maxFreq by scanning all 26 counts every step (an approximate running max, never decremented, is sufficient and correct here).

Interview Follow-up Questions

• Why is it correct to never decrease maxFreq even after shrinking the window?

• How would this change with a lowercase + uppercase mixed alphabet?

Clean Python Solution

```
def character_replacement(s, k):
    counts = {}
    left = 0
    max_freq = 0
    best = 0
    for right, ch in enumerate(s):
        counts[ch] = counts.get(ch, 0) + 1
        max_freq = max(max_freq, counts[ch])
        window_size = right - left + 1
        if window_size - max_freq > k:
            counts[s[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best
```

KEY TAKEAWAY

A window is valid as long as (window size − dominant count) stays within budget — track the dominant count, don't recompute it.

QUESTION 16 OF 75

Hard

Minimum Window Substring

Pattern: Sliding Window (need/have counters)

WHY THIS QUESTION MATTERS

The hardest common sliding-window problem — teaches the need/have counter template used in many string-matching questions.

Problem Statement

Given strings s and t, return the minimum window substring of s that contains every character of t (including duplicates). Return "" if no such substring exists.

Example

```
Input: s = "ADOBECODEBANC", t = "ABC"
→ Output: "BANC"
```

Constraints

1 ≤ s.length, t.length ≤ 10^5 · s and t consist of English letters

Brute-Force Approach

Check every substring of s for containing all of t's characters: O(n³) or O(n²) with counting.

Optimal Approach

Two pointers with a 'need' count map built from t and a 'have' count of the current window. Expand right until the window satisfies all needs, then shrink left as far as possible while still valid, recording the smallest valid window throughout.

Step-by-Step Intuition

Track how many distinct required characters are currently satisfied ('formed' counter). Only when formed == required does shrinking make sense — grow to find validity, shrink to minimize, alternating like a caterpillar.

Dry Run

`s="ADOBECODEBANC", t="ABC". need={A:1,B:1,C:1}. Expand right until window "ADOBEC" satisfies all three — formed=3. Shrink left: removing 'A' breaks it, so record window length 6 first, then continue expanding/shrinking — eventually the tightest valid window found is "BANC" (length 4).`

Time Complexity

O(|s| + |t|) — each pointer moves forward at most |s| times.

Space Complexity

O(|s| + |t|) for the count maps (bounded by alphabet size in practice, O(1) if fixed alphabet).

Common Mistakes

Recomputing 'does window contain all of t' from scratch each time (must be an incremental 'formed' counter); not handling duplicate characters in t correctly (need exact counts, not just presence).

Interview Follow-up Questions

• How would you return all minimum windows, not just one?

• What if t could be empty?

Clean Python Solution

```
from collections import Counter
def min_window(s, t):
    if not s or not t:
        return ""
    need = Counter(t)
    required = len(need)
    window_counts = {}
    formed = 0
    left = 0
    best_len, best_l, best_r = float('inf'), 0, 0
    for right, ch in enumerate(s):
        window_counts[ch] = window_counts.get(ch, 0) + 1
        if ch in need and window_counts[ch] == need[ch]:
            formed += 1
        while formed == required:
            if right - left + 1 < best_len:
                best_len = right - left + 1
                best_l, best_r = left, right
            left_ch = s[left]
            window_counts[left_ch] -= 1
            if left_ch in need and window_counts[left_ch] < need[left_ch]:
                formed -= 1
            left += 1
    return "" if best_len == float('inf') else s[best_l:best_r + 1]
```

KEY TAKEAWAY

Track validity with an incremental 'formed vs required' counter — never re-scan the whole window to check a condition.

QUESTION 17 OF 75

Hard

Sliding Window Maximum

Pattern: Monotonic Deque

WHY THIS QUESTION MATTERS

Introduces the monotonic deque — the sliding-window analogue of a monotonic stack.

Problem Statement

Given an array nums and a window size k, return an array of the maximum value in each sliding window as it moves from left to right across the array.

Example

`Input: nums = [1,3,-1,-3,5,3,6,7], k = 3 → Output: [3,3,5,5,6,7]`

Constraints

1 ≤ nums.length ≤ 10^5 · 1 ≤ k ≤ nums.length

Brute-Force Approach

For each window, scan all k elements to find the max: O(n·k).

Optimal Approach

Maintain a deque of indices whose values are in decreasing order. For each new element, pop smaller values from the back (they can never be the max again), push the new index, and pop the front if it's outside the window. The front of the deque is always the current max.

Step-by-Step Intuition

An element can be discarded forever once a LARGER element appears after it within reach — it will never be the max of any future window. The deque keeps only 'still possibly useful' candidates in decreasing order.

> **Dry Run**
>
> `nums=[1,3,-1,-3,5,3,6,7], k=3. Process 1,3 — pop 1 (smaller), deque=[3]. Process -1 — deque=[3,-1]. Window [1,3,-1] complete, front value=3 → output 3. Slide: process -3 — deque=[3,-1,-3], front still index of 3 → output 3. Process 5 — pop -3,-1,3 (all smaller), deque=[5] → output 5. Continue similarly → [3,3,5,5,6,7].`

Time Complexity

`O(n) — each index is pushed and popped from the deque at most once.`

Space Complexity

`O(k) for the deque.`

Common Mistakes

Storing values instead of indices in the deque (can't tell when an element falls outside the window); using a max-heap (works but O(n log n), not optimal).

Interview Follow-up Questions

- Could you solve this with a balanced BST or heap instead — what's the time complexity trade-off?
- How would you track the sliding window minimum instead?

Clean Python Solution

```
from collections import deque
def max_sliding_window(nums, k):
    dq = deque()
    result = []
    for i, num in enumerate(nums):
        while dq and nums[dq[-1]] < num:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k:
            dq.popleft()
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result
```

KEY TAKEAWAY

A monotonic deque of indices gives O(1) amortized access to a sliding window's max (or min).

CATEGORY 4 OF 14

Binary Search

Not just for sorted arrays — also for searching an answer space (Binary Search on Answer).

6 questions in this section

QUESTION 18 OF 75

Easy

Binary Search

Pattern: Binary Search (classic)

WHY THIS QUESTION MATTERS

The template every other binary search question builds on — get the boundary conditions perfect here.

Problem Statement

Given a sorted array of distinct integers nums and a target, return the index of target, or -1 if it's not present. Must run in O(log n).

Example

`Input: nums = [-1,0,3,5,9,12], target = 9 → Output: 4`

Constraints

1 ≤ nums.length ≤ 10^4 · nums sorted ascending, distinct values

Brute-Force Approach

Linear scan: O(n).

Optimal Approach

Maintain low/high pointers over the sorted range; compare the midpoint to target; discard the half that can't contain it; repeat.

Step-by-Step Intuition

Sortedness means comparing to the midpoint tells you definitively which half to discard — each comparison halves the remaining search space.

Dry Run

`nums=[-1,0,3,5,9,12], target=9. low=0,high=5,mid=2(val 3). 3<9 → low=3. low=3,high=5,mid=4(val 9). Found → return 4.`

Time Complexity

`O(log n).`

Space Complexity

`O(1) iterative (O(log n) if written recursively).`

Common Mistakes

Using mid = (low+high)/2 without worrying about overflow in other languages; off-by-one in the loop condition (low <= high vs low < high) causing infinite loops or missed elements.

Interview Follow-up Questions

• How do you adapt this to find the leftmost or rightmost occurrence when duplicates exist?

• How would you binary search on a 2D sorted matrix?

Clean Python Solution

```
def binary_search(nums, target):
    low, high = 0, len(nums) - 1
    while low <= high:
        mid = (low + high) // 2
        if nums[mid] == target:
            return mid
        elif nums[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

KEY TAKEAWAY

Every binary search is: pick a midpoint, discard the half that can't hold the answer, repeat.

QUESTION 19 OF 75

Medium

Search in Rotated Sorted Array

Pattern: Binary Search (rotated)

WHY THIS QUESTION MATTERS

Teaches how to adapt binary search when the array is 'sorted but shifted' — a very common twist.

Problem Statement

An ascending array is rotated at an unknown pivot. Given the rotated nums and a target, return its index, or -1 if absent, in O(log n).

Example

`Input: nums = [4,5,6,7,0,1,2], target = 0 → Output: 4`

Constraints

1 ≤ nums.length ≤ 5000 · all values distinct · nums is a rotation of an ascending array

Brute-Force Approach

Linear scan for target: O(n).

Optimal Approach

At each midpoint, determine which half (left or right of mid) is properly sorted, then check if target lies within that sorted half's range to decide which side to discard.

Step-by-Step Intuition

Even though the whole array isn't sorted, at least one half of any split always IS sorted — identify that half and use its range to decide where the target could be.

Dry Run

`nums=[4,5,6,7,0,1,2], target=0. low=0,high=6,mid=3(val7). Left half [4..7] is sorted (nums[low]=4<=nums[mid]=7). Is target(0) in [4,7]? No → search right half: low=4. low=4,high=6,mid=5(val1). Right half [1,2] sorted, target 0 not in [1,2] → search left: high=4. low=4,high=4,mid=4(val0) → found, return 4.`

Time Complexity
`O(log n).`

Space Complexity
`O(1).`

Common Mistakes

Forgetting to handle the case where low == mid (single element range); comparing target against the wrong half's bounds; not handling duplicates (which breaks this approach — needs linear fallback).

Interview Follow-up Questions

• What if the array contains duplicate values (Search in Rotated Sorted Array II)?

• How would you find the rotation pivot index itself?

Clean Python Solution

```
def search_rotated(nums, target):
    low, high = 0, len(nums) - 1
    while low <= high:
        mid = (low + high) // 2
        if nums[mid] == target:
            return mid
        if nums[low] <= nums[mid]:
            if nums[low] <= target < nums[mid]:
                high = mid - 1
            else:
                low = mid + 1
        else:
            if nums[mid] < target <= nums[high]:
                low = mid + 1
            else:
                high = mid - 1
    return -1
```

KEY TAKEAWAY

When an array isn't fully sorted, check which half IS sorted at each step — that half's bounds tell you where to search.

QUESTION 20 OF 75

Medium

Find Minimum in Rotated Sorted Array

Pattern: Binary Search (rotated)

WHY THIS QUESTION MATTERS

A cleaner variant of rotated search that isolates the 'find the pivot' logic on its own.

Problem Statement

Given a rotated ascending array with no duplicates, find the minimum element in O(log n).

Example

```
Input: nums = [4,5,6,7,0,1,2] →
Output: 0
```

Constraints

1 ≤ nums.length ≤ 5000 · all values unique · array was originally sorted ascending, then rotated

Brute-Force Approach

Linear scan for the minimum: O(n).

Optimal Approach

Binary search comparing nums[mid] to nums[high]. If nums[mid] > nums[high], the minimum is to the right of mid; otherwise it's at mid or to the left.

Step-by-Step Intuition

The minimum is the one 'break point' where the ascending order resets. Comparing mid to the rightmost element tells you which side that break point is on.

Dry Run

```
nums=[4,5,6,7,0,1,2]. low=0,high=6,mid=3(7). nums[mid]=7 > nums[high]=2 → min is right of mid → low=4. low=4,high=6,mid=5(1). nums[mid]=1 <= nums[high]=2 → min at mid or left → high=5. low=4,high=5,mid=4(0). nums[mid]=0<=nums[high]=1 → high=4. low==high=4 → return nums[4]=0.
```

Time Complexity

O(log n).

Space Complexity

O(1).

Common Mistakes

Comparing to nums[low] instead of nums[high] (breaks the logic in several rotation cases); not converging the loop condition to low < high correctly.

Interview Follow-up Questions

• How does the approach change if duplicates are allowed?

• Can you find the number of times the array was rotated?

Clean Python Solution

```python
def find_min(nums):
    low, high = 0, len(nums) - 1
    while low < high:
        mid = (low + high) // 2
        if nums[mid] > nums[high]:
            low = mid + 1
        else:
            high = mid
    return nums[low]
```

KEY TAKEAWAY

Compare the midpoint to the RIGHT boundary (not the left) to locate a single break point in a rotated sequence.

QUESTION 21 OF 75

Medium

Koko Eating Bananas

Pattern: Binary Search on Answer

WHY THIS QUESTION MATTERS

The defining Binary Search on Answer question — the answer space (not the input array) is what you binary search over.

Problem Statement

Koko has piles of bananas and h hours before the guards return. Each hour she picks one pile and eats up to k bananas from it (if the pile has fewer than k, she finishes it and stops for that hour). Find the minimum integer eating speed k such that she can eat all bananas within h hours.

Example

```
Input: piles = [3,6,7,11], h = 8 →
Output: 4
```

Constraints

1 ≤ piles.length ≤ 10^4 · piles.length ≤ h ≤ 10^9 · 1 ≤ piles[i] ≤ 10^9

Brute-Force Approach

Try every speed k from 1 upward, checking feasibility each time: O(max(piles) · n).

Optimal Approach

Binary search k between 1 and max(piles). For each candidate k, compute hours needed = sum(ceil(pile / k) for each pile); if hours ≤ h, k is feasible (try smaller); else try larger.

Step-by-Step Intuition

Feasibility is monotonic: if speed k works, every speed greater than k also works. That monotonic 'yes region / no region' boundary is exactly what binary search finds — you're searching over possible ANSWERS, not array indices.

Dry Run

```
piles=[3,6,7,11], h=8. low=1,high=11. mid=6:
hours=ceil(3/6)+ceil(6/6)+ceil(7/6)+ceil(11/6)=1+1+2+2=6≤8 → feasible, try
smaller: high=6. mid=3: hours=1+2+3+4=10>8 → infeasible, low=4. mid=5:
hours=1+2+2+3=8≤8 → high=5. mid=4: hours=1+2+2+3=8≤8 → high=4. low==high=4 →
answer 4.
```

Time Complexity

O(n · log(max(piles))) — binary search over speed, each check is O(n).

Space Complexity

O(1).

Common Mistakes

Forgetting ceiling division (a partial pile still costs a full hour); searching the wrong bounds (low should be 1, not 0, since speed 0 is invalid).

Interview Follow-up Questions

• How would you adapt this to 'minimum days to ship packages within weight capacity D' (an isomorphic problem)?

• What if k must be one of a fixed set of allowed speeds?

Clean Python Solution

```
import math
def min_eating_speed(piles, h):
    low, high = 1, max(piles)
    while low < high:
        mid = (low + high) // 2
        hours = sum(math.ceil(pile / mid) for pile in piles)
        if hours <= h:
            high = mid
        else:
            low = mid + 1
    return low
```

KEY TAKEAWAY

When feasibility of an answer is monotonic (works → everything bigger also works), binary search the answer space directly.

QUESTION 22 OF 75

Medium

Find First and Last Position of Element in Sorted Array

Pattern: Binary Search (leftmost/rightmost bound)

WHY THIS QUESTION MATTERS

Teaches the two-variant binary search template (find leftmost / find rightmost) needed whenever duplicates exist.

Problem Statement

Given a sorted array nums and a target, find the starting and ending index of target's occurrences. Return [-1,-1] if target is not found. Must run in O(log n).

Example

`Input: nums = [5,7,7,8,8,10], target = 8 → Output: [3,4]`

Constraints

0 ≤ nums.length ≤ 10^5 · nums sorted ascending, may contain duplicates

Brute-Force Approach

Linear scan to find first and last matching index: O(n).

Optimal Approach

Run two modified binary searches: one that keeps moving left even after finding target (to find the leftmost occurrence), another that keeps moving right (to find the rightmost).

Step-by-Step Intuition

A normal binary search stops at ANY match. To find a boundary, don't stop at a match — keep narrowing toward the edge you want, recording the best match seen so far.

Dry Run

`nums=[5,7,7,8,8,10], target=8. Leftmost search: mid narrows down, each time nums[mid]==8 records index and moves high=mid-1 to look further left — converges to index 3. Rightmost search similarly converges to index 4. Output [3,4].`

Time Complexity

`O(log n) — two binary searches.`

Space Complexity

`O(1).`

Common Mistakes

Writing one binary search and trying to reuse it for both bounds without changing the tie-breaking direction; off-by-one errors when narrowing after a match.

Interview Follow-up Questions

- How would you count the total occurrences of target using these two bounds?
- Can you generalize this to find the k-th occurrence?

Clean Python Solution

```
def search_range(nums, target):
    def find_bound(is_first):
        low, high = 0, len(nums) - 1
        result = -1
        while low <= high:
            mid = (low + high) // 2
            if nums[mid] == target:
                result = mid
                if is_first:
                    high = mid - 1
                else:
                    low = mid + 1
            elif nums[mid] < target:
                low = mid + 1
            else:
                high = mid - 1
        return result
    return [find_bound(True), find_bound(False)]
```

KEY TAKEAWAY

To find a boundary (not just any match), keep searching past a hit in the direction of the boundary you want.

QUESTION 23 OF 75

Hard

Median of Two Sorted Arrays

Pattern: Binary Search (partition)

WHY THIS QUESTION MATTERS

A landmark hard problem — shows binary search can partition two arrays simultaneously, not just search one.

Problem Statement

Given two sorted arrays nums1 and nums2 of size m and n, return the median of the combined sorted array in O(log(min(m,n))) time.

Example

```
Input: nums1 = [1,3], nums2 = [2] →
Output: 2.0
```

Constraints

0 ≤ m, n ≤ 1000 · 1 ≤ m+n ≤ 2000 · arrays sorted ascending

Brute-Force Approach

Merge both arrays and take the middle element(s): O(m+n) time, O(m+n) space.

Optimal Approach

Binary search on a PARTITION index in the smaller array. For each partition, compute the matching partition in the other array such that left-side count == right-side count, then check the boundary values line up correctly (max of lefts ≤ min of rights). Adjust the partition based on which boundary condition fails.

Step-by-Step Intuition

The median splits the combined array into two equal halves. Binary search directly for the partition point in the smaller array where 'everything to the left of both partitions' is ≤ 'everything to the right of both partitions' — no merging needed.

Dry Run

```
nums1=[1,3], nums2=[2]. Total=3, half=2. Binary search partition in nums1
(smaller). Try partitionX=1: left1=1(val1), right1=3. partitionY=1:
left2=2(val2), right2=+inf. Check max(left1,left2)=2 <= min(right1,right2)=3 →
valid partition. Combined length odd(3) → median = max(left1,left2) = 2.0.
```

Time Complexity

```
O(log(min(m,n))) — binary search
only over the smaller array.
```

Space Complexity

```
O(1).
```

Common Mistakes

Binary searching the larger array (still correct but slower — interviewers want the smaller one for the tight bound); mishandling empty-partition edge cases (use ±infinity sentinels).

Interview Follow-up Questions

• How would you find the k-th smallest element of two sorted arrays generally (this is a special case of that)?

• What if the arrays could contain duplicates — does anything change?

Clean Python Solution

```
def find_median_sorted_arrays(nums1, nums2):
    if len(nums1) > len(nums2):
        nums1, nums2 = nums2, nums1
    m, n = len(nums1), len(nums2)
    low, high = 0, m
    half = (m + n + 1) // 2
    while low <= high:
        i = (low + high) // 2
        j = half - i
        left1 = nums1[i - 1] if i > 0 else float('-inf')
        right1 = nums1[i] if i < m else float('inf')
        left2 = nums2[j - 1] if j > 0 else float('-inf')
        right2 = nums2[j] if j < n else float('inf')
        if left1 <= right2 and left2 <= right1:
            if (m + n) % 2 == 0:
                return (max(left1, left2) + min(right1, right2)) / 2
            return float(max(left1, left2))
        elif left1 > right2:
            high = i - 1
        else:
            low = i + 1
    return 0.0
```

KEY TAKEAWAY

Binary search doesn't need to search 'in' a single array — it can search over a space of valid partitions or configurations.

CATEGORY 5 OF 14

Strings

String-specific manipulation: parsing, expansion, and encoding tricks interviewers love.

5 questions in this section

QUESTION 24 OF 75

Easy

Longest Common Prefix

Pattern: String Scanning

WHY THIS QUESTION MATTERS

A gentle string warm-up that teaches vertical (column-by-column) scanning across multiple strings.

Problem Statement

Given an array of strings strs, find the longest common prefix shared by all of them. Return "" if there is none.

Example

`Input: strs = ["flower","flow","flight"] → Output: "fl"`

Constraints

1 ≤ strs.length ≤ 200 · 0 ≤ strs[i].length ≤ 200

Brute-Force Approach

Compare every string pairwise character by character, tracking the shortest common prefix seen: O(S) where S is total characters, but implemented clumsily can be worse.

Optimal Approach

Scan column by column: for each character position, check if all strings share the same character at that index; stop at the first mismatch or the shortest string's end.

Step-by-Step Intuition

A common prefix can never be longer than the shortest string, and it breaks at the first column where any string disagrees — so scan vertically and stop at the first disagreement.

Dry Run

`strs=["flower","flow","flight"]. col0: f,f,f match. col1: l,l,l match. col2: o,o,i — mismatch → stop. Prefix="fl".`

Time Complexity

`O(S) where S is the sum of all character lengths (worst case).`

Space Complexity

`O(1) extra beyond the output.`

Common Mistakes

Not handling an empty input list; not stopping as soon as any string is exhausted; comparing strings in a way that's O(n²) unnecessarily.

Interview Follow-up Questions

• How would you find the longest common suffix instead?

• Can you solve this with a Trie for repeated queries?

Clean Python Solution

```
def longest_common_prefix(strs):
    if not strs:
        return ""
    for i, ch in enumerate(strs[0]):
        for s in strs[1:]:
            if i >= len(s) or s[i] != ch:
                return strs[0][:i]
    return strs[0]
```

KEY TAKEAWAY

When comparing many strings, scan column by column and bail out at the first disagreement.

QUESTION 25 OF 75

Medium

Longest Palindromic Substring

Pattern: Expand Around Center

WHY THIS QUESTION MATTERS

The classic 'expand around center' pattern — a simpler alternative to DP for palindrome problems.

Problem Statement

Given a string s, return the longest palindromic substring in s.

Example

`Input: s = "babad" → Output: "bab" (or "aba")`

Constraints

1 ≤ s.length ≤ 1000 · s consists of digits and English letters

Brute-Force Approach

Check every substring for being a palindrome: O(n³), or O(n²) with DP.

Optimal Approach

For each index, expand outward in both directions treating it as a palindrome center — once for odd-length palindromes (single center) and once for even-length (between two characters). Track the longest found.

Step-by-Step Intuition

Every palindrome has a center (a single character, or the gap between two characters). Instead of checking all O(n²) substrings, try expanding from each of the 2n-1 possible centers and stop as soon as the expansion breaks.

Dry Run

s="babad". Center at index1('a'): expand — s[0]='b'==s[2]='b' → "bab" (len3), expand further out of bounds → stop. Center at index2('b') (as odd center too): "aba" also len 3 found later. Longest found = "bab" (first found, length 3).

Time Complexity

$O(n^2)$ — n centers, each expansion up to O(n).

Space Complexity

O(1) extra.

Common Mistakes

Forgetting even-length palindromes (centers between characters); off-by-one in the expansion bounds check.

Interview Follow-up Questions

- Can you solve this in O(n) with Manacher's Algorithm?
- How would you count all palindromic substrings instead of finding just the longest?

Clean Python Solution

```
def longest_palindrome(s):
    def expand(l, r):
        while l >= 0 and r < len(s) and s[l] == s[r]:
            l -= 1
            r += 1
        return s[l + 1:r]
    best = ""
    for i in range(len(s)):
        odd = expand(i, i)
        even = expand(i, i + 1)
        for cand in (odd, even):
            if len(cand) > len(best):
                best = cand
    return best
```

KEY TAKEAWAY

Expanding outward from every possible center is a simple O(n²) alternative to palindrome DP.

QUESTION 26 OF 75

Medium

Palindromic Substrings

Pattern: Expand Around Center

WHY THIS QUESTION MATTERS

Reinforces expand-around-center with a counting (not just finding) objective.

Problem Statement

Given a string s, return the number of palindromic substrings in it (different positions count separately even if the substring text repeats).

Example

Constraints

Input: s = "aaa" → Output: 6 ("a","a","a","aa","aa","aaa")

1 ≤ s.length ≤ 1000 · lowercase English letters

Brute-Force Approach

Check every substring for being a palindrome: O(n³).

Optimal Approach

For each of the 2n-1 centers, expand outward and count every successful expansion as one more palindrome.

Step-by-Step Intuition

Every valid expansion step from a center IS a distinct palindromic substring — so counting successful expansions directly counts palindromes, no separate verification needed.

> **Dry Run**
> s="aaa". Center0('a'): expands once (just "a") — count 1. Center1('a'): expands to "a", then "aaa" — count 2. Center2('a'): "a" — count 1. Even centers (0,1): "aa" — count1. Even centers(1,2): "aa" — count1. Total=1+2+1+1+1=6.

Time Complexity
O(n²).

Space Complexity
O(1) extra.

Common Mistakes

Forgetting even-length centers; double counting or under counting by mismanaging the expansion loop's stopping condition.

Interview Follow-up Questions

- Can you do this in O(n) using Manacher's Algorithm?
- How would you list the substrings themselves, not just the count?

Clean Python Solution

```
def count_substrings(s):
    def expand(l, r):
        count = 0
        while l >= 0 and r < len(s) and s[l] == s[r]:
            count += 1
            l -= 1
            r += 1
        return count
    total = 0
    for i in range(len(s)):
        total += expand(i, i)
        total += expand(i, i + 1)
    return total
```

KEY TAKEAWAY

Counting successful center-expansions is equivalent to counting all palindromic substrings.

QUESTION 27 OF 75

Medium

String to Integer (atoi)

Pattern: Careful Parsing / Edge-Case Enumeration

WHY THIS QUESTION MATTERS

Tests disciplined edge-case handling — a realistic 'implement a library function correctly' interview question.

Problem Statement

Implement atoi to convert a string to a 32-bit signed integer, following: skip leading whitespace, an optional +/- sign, then digits until a non-digit; clamp to the 32-bit signed integer range; return 0 if no valid conversion exists.

Example

`Input: s = "   -42" → Output: -42`
`Input: s = "4193 with words" → Output: 4193`

Constraints

0 ≤ s.length ≤ 200 · s may contain letters, digits, spaces, '+', '-', '.'

Brute-Force Approach

N/A — this problem is inherently about correct sequential parsing, not algorithmic optimization.

Optimal Approach

Walk the string once: skip leading spaces, capture an optional sign, accumulate digits while clamping against INT_MAX/INT_MIN as you go, then stop at the first non-digit.

Step-by-Step Intuition

Treat it as a small state machine: whitespace → sign → digits → stop. Clamp during accumulation (not after) to avoid overflow in languages with fixed-width integers.

Dry Run

`s="  -42". Skip 3 spaces. See '-' → sign=-1. Accumulate digits '4' then '2' → num=42. End of string → result = -1*42 = -42.`

Time Complexity

`O(n) — single pass.`

Space Complexity

`O(1).`

Common Mistakes

Not clamping to INT_MIN/INT_MAX; not stopping at the first non-digit character (e.g. "4193 with words" must stop at the space); mishandling '+' vs '-' vs no sign; not skipping leading whitespace only (not embedded whitespace).

Interview Follow-up Questions

- How would you also support scientific notation like "1e10"?
- What would change if this needed to support 64-bit integers instead?

Clean Python Solution

```
def my_atoi(s):
    i, n = 0, len(s)
    while i < n and s[i] == ' ':
        i += 1
    if i == n:
        return 0
    sign = 1
    if s[i] in '+-':
        sign = -1 if s[i] == '-' else 1
        i += 1
    num = 0
    INT_MAX, INT_MIN = 2**31 - 1, -2**31
    while i < n and s[i].isdigit():
        num = num * 10 + int(s[i])
        i += 1
        if sign * num > INT_MAX:
            return INT_MAX
        if sign * num < INT_MIN:
            return INT_MIN
    return sign * num
```

KEY TAKEAWAY

When a problem is 'implement this spec correctly', enumerate every edge case as an explicit state before writing code.

QUESTION 28 OF 75

Medium

Encode and Decode Strings

Pattern: Delimiter Design

WHY THIS QUESTION MATTERS

A design-oriented string question — teaches length-prefixing, a real technique used in network protocols.

Problem Statement

Design an algorithm to encode a list of strings into a single string, and decode that string back into the original list of strings. Any characters (including delimiters) may appear inside the strings.

Example

`Input: ["hello","world"] → encode → "5#hello5#world" → decode → ["hello","world"]`

Constraints

strings may contain any character, including the delimiter you might otherwise choose · must round-trip exactly

Brute-Force Approach

Join with a common delimiter like a comma — breaks if a string itself contains a comma.

Optimal Approach

Prefix each string with its length and a non-ambiguous separator (e.g. "5#hello"), since the length is always a clean, unambiguous marker regardless of what characters follow it.

Step-by-Step Intuition

Any single-character delimiter can collide with the data. But a LENGTH is never ambiguous — encode 'read exactly N characters next' and you never need to worry about what's inside those N characters.

> **Dry Run**
> `Encode ["hello","world"]: "5#hello" + "5#world" = "5#hello5#world". Decode: read digits until '#' → 5, read next 5 chars → "hello", advance; read digits until '#' → 5, read next 5 chars → "world". Result: ["hello","world"].`

Time Complexity
`O(total characters) for both encode and decode.`

Space Complexity
`O(total characters) for the encoded string.`

Common Mistakes

Choosing a fixed delimiter character that could appear in the data (fragile); forgetting to handle empty strings in the list.

Interview Follow-up Questions

• How would you handle extremely large strings where length could exceed normal integer display width?

• Could you design this to be streamable (decode without holding the entire string in memory)?

Clean Python Solution

```python
def encode(strs):
    return ''.join(f"{len(s)}#{s}" for s in strs)

def decode(s):
    result = []
    i = 0
    while i < len(s):
        j = i
        while s[j] != '#':
            j += 1
        length = int(s[i:j])
        start = j + 1
        result.append(s[start:start + length])
        i = start + length
    return result
```

> **KEY TAKEAWAY**
> *Length-prefixing beats delimiter characters whenever the data itself can contain arbitrary characters.*

CATEGORY 6 OF 14

Linked Lists

Pointer manipulation without index access — reversal and fast/slow pointer tricks dominate.

6 questions in this section

QUESTION 29 OF 75

Easy

Reverse Linked List

Pattern: Iterative Pointer Reversal

WHY THIS QUESTION MATTERS

The single most important linked-list primitive — used inside many harder list problems.

Problem Statement

Given the head of a singly linked list, reverse the list and return the new head.

Example

```
Input: 1 -> 2 -> 3 -> nullptr →
Output: 3 -> 2 -> 1 -> nullptr
```

Constraints

0 ≤ number of nodes ≤ 5000

Brute-Force Approach

Push all values onto a stack (or into an array), then rebuild a new list in reverse: O(n) time, O(n) space.

Optimal Approach

Walk the list once with a 'prev' pointer starting at None. At each node, save the next node, point current.next back to prev, then advance both prev and current.

Step-by-Step Intuition

You only need to flip each node's single outgoing pointer to point backward instead of forward — no extra storage required, just careful pointer bookkeeping as you walk.

Dry Run

`1->2->3->None. prev=None,cur=1. next=2, 1.next=None(prev), prev=1,cur=2. next=3, 2.next=1, prev=2,cur=3. next=None, 3.next=2, prev=3,cur=None. Loop ends, return prev=3 → 3->2->1->None.`

Time Complexity

O(n) — single pass.

Space Complexity

O(1) iterative (O(n) call stack if done recursively).

Common Mistakes

Losing the reference to 'next' before reassigning current.next (causes a broken/lost list); returning 'current' instead of 'prev' at the end (current is None when the loop ends).

Interview Follow-up Questions

• Can you write the recursive version, and what's its space trade-off?

• How would you reverse only a sublist between positions m and n?

Clean Python Solution

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def reverse_list(head):
    prev = None
    cur = head
    while cur:
        nxt = cur.next
        cur.next = prev
        prev = cur
        cur = nxt
    return prev
```

KEY TAKEAWAY

Reversing a singly linked list is three pointers (prev, current, next) walked in lockstep — memorize this cold.

QUESTION 30 OF 75

Easy

Linked List Cycle

Pattern: Fast & Slow Pointers

WHY THIS QUESTION MATTERS

The introductory Fast & Slow Pointers (Floyd's Tortoise and Hare) question.

Problem Statement

Given the head of a linked list, determine if it has a cycle (some node's next eventually points back to a previous node).

Example

`Input: 3 -> 2 -> 0 -> -4 -> (back to 2) → Output: true`

Constraints

0 ≤ number of nodes ≤ 10^4

Brute-Force Approach

Store every visited node in a hash set; if you revisit a node, a cycle exists: O(n) time, O(n) space.

Optimal Approach

Use two pointers, slow (moves 1 step) and fast (moves 2 steps). If they ever meet, there's a cycle. If fast reaches the end (None), there's no cycle.

Step-by-Step Intuition

In a cycle, the faster pointer gains one step on the slower pointer every iteration and is trapped inside the same finite loop — it's mathematically guaranteed to lap and meet the slow pointer eventually.

> **Dry Run**
> `3->2->0->-4->(cycle to 2). slow=3,fast=3. Step1: slow=2,fast=0. Step2: slow=0,fast=2(via cycle). Step3: slow=-4,fast=-4 → slow==fast → cycle detected, return True.`

Time Complexity
`O(n).`

Space Complexity
`O(1) — no extra data structure, unlike the hash-set approach.`

Common Mistakes

Not checking fast and fast.next for None before advancing (causes a null pointer error); using a hash set when O(1) space is explicitly requested.

Interview Follow-up Questions

• How would you find the node where the cycle BEGINS (Linked List Cycle II)?

• How would you find the length of the cycle?

Clean Python Solution

```
def has_cycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

KEY TAKEAWAY

A pointer moving twice as fast as another will always catch up inside a cycle — that's the whole trick.

QUESTION 31 OF 75

Easy

Middle of the Linked List

Pattern: Fast & Slow Pointers

WHY THIS QUESTION MATTERS

Shows the fast/slow pointer trick used for something other than cycle detection: finding the midpoint in one pass.

Problem Statement

Given the head of a singly linked list, return the middle node. If there are two middle nodes, return the second one.

Example

```
Input: 1 -> 2 -> 3 -> 4 -> 5 → Output: node with value 3
Input: 1 -> 2 -> 3 -> 4 -> 5 -> 6 → Output: node with value 4
```

Constraints

1 ≤ number of nodes ≤ 100

Brute-Force Approach

Traverse once to count length n, traverse again to node n//2: O(n) time but two passes.

Optimal Approach

Fast and slow pointers both start at head; slow moves 1 step, fast moves 2 steps. When fast reaches the end, slow is at the middle.

Step-by-Step Intuition

By the time the fast pointer (moving 2x speed) has covered the whole list, the slow pointer (moving 1x speed) has covered exactly half of it — a single pass finds the midpoint.

Dry Run

`1->2->3->4->5. slow=1,fast=1. step: slow=2,fast=3. step: slow=3,fast=5. fast.next is None → stop. Return slow=3 (the middle).`

Time Complexity

`O(n) — but a single pass, not two.`

Space Complexity

`O(1).`

Common Mistakes

Using two separate passes when one is expected; getting the 'first vs second middle' convention wrong for even-length lists.

Interview Follow-up Questions

• How would this change if you needed the FIRST middle node for even-length lists instead?

• Can you use this to check if a linked list is a palindrome?

Clean Python Solution

```python
def middle_node(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    return slow
```

KEY TAKEAWAY

Fast/slow pointers find the midpoint of a list in one pass without knowing its length in advance.

QUESTION 32 OF 75

Easy

Merge Two Sorted Lists

Pattern: Two-Pointer List Merge

WHY THIS QUESTION MATTERS

The linked-list analogue of the merge step in merge sort — foundational for Merge K Sorted Lists later.

Problem Statement

Given the heads of two sorted linked lists, merge them into one sorted list by splicing the existing nodes together, and return the head.

Example

```
Input: l1 = 1->2->4, l2 = 1->3->4 →
Output: 1->1->2->3->4->4
```

Constraints

0 ≤ nodes in each list ≤ 50 · both lists sorted ascending

Brute-Force Approach

Extract all values into an array, sort, rebuild a new list: O(n log n) time, O(n) space.

Optimal Approach

Use a dummy head node and a tail pointer. Repeatedly compare the fronts of both lists, splice the smaller node onto the result, and advance that list's pointer. Attach whichever list remains at the end.

Step-by-Step Intuition

Both lists are already sorted, so you never need to look ahead — just always take whichever current node is smaller, exactly like the merge step of merge sort.

Dry Run

`l1=1->2->4, l2=1->3->4. dummy->_. Compare 1,1: take l1's 1 (tie-break arbitrary) → result:1. Compare 2,1: take l2's 1 → result:1->1. Compare 2,3: take 2 → result:...->2. Compare 4,3: take 3 → ...->3. Compare 4,4: take 4, then attach remaining 4 → final: 1->1->2->3->4->4.`

Time Complexity

`O(n + m)` — total nodes in both lists.

Space Complexity

`O(1)` extra — reuses existing nodes (`O(n+m)` if done recursively, due to call stack).

Common Mistakes

Building brand-new nodes instead of splicing existing ones (wastes memory); forgetting to attach the remaining tail of whichever list isn't exhausted first.

Interview Follow-up Questions

- How would you extend this to merge k sorted lists (see Question 49)?
- Can you write this recursively, and what's the trade-off?

Clean Python Solution

```
def merge_two_lists(l1, l2):
    dummy = ListNode()
    tail = dummy
    while l1 and l2:
        if l1.val <= l2.val:
            tail.next = l1
            l1 = l1.next
        else:
            tail.next = l2
            l2 = l2.next
        tail = tail.next
    tail.next = l1 if l1 else l2
    return dummy.next
```

KEY TAKEAWAY

A dummy head node simplifies list-building code by removing all 'is this the first node' special cases.

QUESTION 33 OF 75

Medium

Remove Nth Node From End of List

Pattern: Two Pointers (fixed gap)

WHY THIS QUESTION MATTERS

Shows how a fixed gap between two pointers lets you find a position 'from the end' without knowing the list's length.

Problem Statement

Given the head of a linked list, remove the nth node from the end of the list and return the head, in a single pass.

Example

```
Input: head = 1->2->3->4->5, n = 2 →
Output: 1->2->3->5
```

Constraints

1 ≤ number of nodes ≤ 30 · 0 ≤ n ≤ number of nodes

Brute-Force Approach

Traverse once to find length L, then traverse again to node (L - n - 1) to unlink the target: O(n) but two passes.

Optimal Approach

Use a dummy head. Advance a 'fast' pointer n+1 steps ahead of a 'slow' pointer, then move both together until fast reaches the end — slow now sits right before the node to remove.

Step-by-Step Intuition

Keeping a fixed gap of n+1 nodes between two pointers means that when the front pointer runs out of list, the back pointer is exactly n nodes from the end — achieved in one pass.

> **Dry Run**

head=1->2->3->4->5, n=2. dummy->1->2->3->4->5. fast starts at dummy, advances 3 steps (n+1): fast at node 3. Now move slow(dummy) and fast together until fast is None: slow ends at node 3 (value 3), fast passes 4,5,None. slow.next(4) is the node to remove → slow.next = slow.next.next(5). Result: 1->2->3->5.

Time Complexity
O(n) — single pass.

Space Complexity
O(1).

Common Mistakes

Off-by-one on how far ahead to advance the fast pointer (should be n+1 steps from the dummy, not n); forgetting the dummy head, which is what makes removing the actual head node (n == length) work cleanly.

Interview Follow-up Questions

• How would you solve this in two passes if single-pass wasn't required?

• Can you remove the nth node from the START just as easily — why or why not?

Clean Python Solution

```
def remove_nth_from_end(head, n):
    dummy = ListNode(0, head)
    fast = slow = dummy
    for _ in range(n + 1):
        fast = fast.next
    while fast:
        fast = fast.next
        slow = slow.next
    slow.next = slow.next.next
    return dummy.next
```

KEY TAKEAWAY

A fixed-gap two-pointer walk finds a position relative to the END of a list without a length precomputation.

QUESTION 34 OF 75

Medium

Reorder List

Pattern: Fast/Slow + Reverse + Merge

WHY THIS QUESTION MATTERS

Combines three earlier patterns (find middle, reverse a list, merge two lists) into one problem — a great 'compose your toolkit' exercise.

Problem Statement

Given the head of a singly linked list L0 -> L1 -> ... -> Ln, reorder it in place to: L0 -> Ln -> L1 -> Ln-1 -> L2 -> Ln-2 -> ... You may not modify node values, only pointers.

Example
Input: 1->2->3->4 → Output: 1->4->2->3

Constraints
1 ≤ number of nodes ≤ 5×10^4

Brute-Force Approach

Copy all nodes into an array/list, then use two index pointers to rebuild the links in the desired interleaved order: O(n) time, O(n) space.

Optimal Approach

1) Find the middle with fast/slow pointers. 2) Reverse the second half in place. 3) Merge the first half and reversed second half by alternating nodes.

Step-by-Step Intuition

The target order is 'first half, interleaved with the reversed second half' — so split the list in half, reverse the tail, and zipper-merge the two halves together.

> **Dry Run**
>
> `1->2->3->4. Middle found at node 2 (slow) via fast/slow — split into 1->2 and 3->4. Reverse second half: 4->3. Merge alternating: take 1, take 4, take 2, take 3 → 1->4->2->3.`

Time Complexity

`O(n) — each of the three steps is linear.`

Space Complexity

`O(1) extra — all done via pointer rewiring, no extra array.`

Common Mistakes

Not cleanly splitting the list into two halves (off-by-one on where the split happens for odd vs even length); forgetting to terminate the first half's list with None before merging (creates a cycle).

Interview Follow-up Questions

- How would you verify a linked list is a palindrome using the same 'find middle + reverse half' toolkit?
- What changes for a doubly linked list?

Clean Python Solution

```
def reorder_list(head):
    if not head or not head.next:
        return
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
    second = slow.next
    slow.next = None
    prev = None
    while second:
        nxt = second.next
        second.next = prev
        prev = second
        second = nxt
    first, second = head, prev
    while second:
        n1, n2 = first.next, second.next
        first.next = second
        second.next = n1
        first, second = n1, n2
```

KEY TAKEAWAY

Hard list problems often decompose into a sequence of easier list primitives you already know — find middle, reverse, merge.

CATEGORY 7 OF 14

Stack & Queue

LIFO/FIFO structures that power parsing, monotonic stacks, and evaluation problems.

5 questions in this section

QUESTION 35 OF 75

Easy

Valid Parentheses

Pattern: Stack (matching pairs)

WHY THIS QUESTION MATTERS

The canonical stack problem — matching/nesting structure always maps to a stack.

Problem Statement

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid (every open bracket is closed by the same type in the correct order).

Example

```
Input: s = "()[]{}" → Output: true
Input: s = "(]" → Output: false
```

Constraints

1 ≤ s.length ≤ 10^4 · s consists only of bracket characters

Brute-Force Approach

Repeatedly find and remove adjacent matching pairs like "()" from the string until no more can be removed, then check if empty: O(n²) due to repeated scanning.

Optimal Approach

Push open brackets onto a stack. On a closing bracket, pop the stack and check it matches the expected open bracket type; if not (or stack is empty), the string is invalid. At the end, the stack must be empty.

Step-by-Step Intuition

The most recently opened bracket must be the next one closed — that 'last opened, first closed' behavior is exactly a stack (LIFO).

Dry Run

`s="()[]{}": '(' push. ')' pop '(' — match. '[' push. ']' pop '[' — match. '{' push. '}' pop '{' — match. Stack empty at end → True.`

Time Complexity

O(n).

Space Complexity

O(n) worst case (all opening brackets).

Common Mistakes

Popping from an empty stack without checking (causes a crash on inputs like ")("); forgetting the final check that the stack is empty (catches unclosed brackets like "((((").

Interview Follow-up Questions

• How would you find the position of the first invalid character?

• What's the minimum number of insertions to make the string valid?

Clean Python Solution

```
def is_valid(s):
    pairs = {')': '(', ']': '[', '}': '{'}
    stack = []
    for ch in s:
        if ch in pairs:
            if not stack or stack.pop() != pairs[ch]:
                return False
        else:
            stack.append(ch)
    return not stack
```

KEY TAKEAWAY

Any 'most recent must be resolved first' matching problem is a stack problem.

QUESTION 36 OF 75

Medium

Min Stack

Pattern: Stack (auxiliary tracking)

WHY THIS QUESTION MATTERS

Teaches the 'auxiliary stack' trick — track extra state (running min) alongside a stack in O(1).

Problem Statement

Design a stack that supports push, pop, top, and retrieving the minimum element, all in O(1) time.

Example

`push(-2), push(0), push(-3), getMin() → -3, pop(), top() → 0, getMin() → -2`

Constraints

methods called: -2^31 ≤ val ≤ 2^31-1 · pop/top/getMin always called on a non-empty stack

Brute-Force Approach

Recompute the minimum by scanning the whole stack every time getMin is called: O(n) per call.

Optimal Approach

Maintain a second stack that tracks the minimum value at each corresponding depth of the main stack; push/pop both stacks in lockstep.

Step-by-Step Intuition

Instead of recomputing the min, just remember what the min WAS at every point in the stack's history — a parallel min-stack gives O(1) access to 'the min if I were to pop back to here'.

Dry Run

push(-2): main=[-2], minStack=[-2]. push(0): main=[-2,0], minStack=[-2,-2] (min stays -2). push(-3): main=[-2,0,-3], minStack=[-2,-2,-3]. getMin()=-3. pop(): main=[-2,0], minStack=[-2,-2]. getMin()=-2.

Time Complexity
O(1) for every operation.

Space Complexity
O(n) for the auxiliary min-stack.

Common Mistakes

Storing the min only once instead of once per push (breaks when popping); forgetting to pop the min-stack in lockstep with the main stack.

Interview Follow-up Questions

• Can you do this with O(1) extra space (not a full second stack) using difference encoding?

• How would you also support getMax() simultaneously?

Clean Python Solution

```
class MinStack:
    def __init__(self):
        self.stack = []
        self.min_stack = []

    def push(self, val):
        self.stack.append(val)
        m = val if not self.min_stack else min(val, self.min_stack[-1])
        self.min_stack.append(m)

    def pop(self):
        self.stack.pop()
        self.min_stack.pop()

    def top(self):
        return self.stack[-1]

    def get_min(self):
        return self.min_stack[-1]
```

KEY TAKEAWAY

A parallel auxiliary stack lets you answer 'what was true at this depth' in O(1), without recomputation.

QUESTION 37 OF 75

Medium

Evaluate Reverse Polish Notation

Pattern: Stack (expression evaluation)

WHY THIS QUESTION MATTERS

Shows how a stack naturally evaluates postfix expressions — the basis of calculator/compiler-style questions.

Problem Statement

Evaluate the value of an arithmetic expression given in Reverse Polish Notation (postfix), where tokens are integers or one of + - * /.

Example

```
Input: tokens = ["2","1","+","3","*"]
→ Output: 9  ((2+1)*3)
```

Constraints

1 ≤ tokens.length ≤ 10^4 · division truncates toward zero · the expression is always valid

Brute-Force Approach

N/A — postfix evaluation is inherently a stack-based process; there isn't a meaningfully different brute force.

Optimal Approach

Push numbers onto a stack. When an operator is seen, pop the top two numbers, apply the operator, and push the result back.

Step-by-Step Intuition

Postfix notation is DESIGNED so that operands always appear before their operator, meaning the two most recently pushed values are always exactly the operator's operands.

Dry Run

```
tokens=["2","1","+","3","*"]. push 2 → [2]. push 1 → [2,1]. '+' → pop 1,2 →
2+1=3, push → [3]. push 3 → [3,3]. '*' → pop 3,3 → 3*3=9, push → [9]. Result=9.
```

Time Complexity

```
O(n) — each token processed once.
```

Space Complexity

```
O(n) for the stack.
```

Common Mistakes

Popping operands in the wrong order for non-commutative operators (subtraction/division — the FIRST popped value is the right operand, not the left); not truncating division toward zero as specified (Python's // rounds toward negative infinity, requiring int(a/b) instead).

Interview Follow-up Questions

• How would you evaluate a standard infix expression with operator precedence instead?

• How would you support additional operators like exponentiation?

Clean Python Solution

```
def eval_rpn(tokens):
    stack = []
    ops = {'+', '-', '*', '/'}
    for token in tokens:
        if token in ops:
            b = stack.pop()
            a = stack.pop()
            if token == '+':
                stack.append(a + b)
            elif token == '-':
                stack.append(a - b)
            elif token == '*':
                stack.append(a * b)
            else:
                stack.append(int(a / b))
        else:
            stack.append(int(token))
    return stack[0]
```

KEY TAKEAWAY

Postfix expressions evaluate directly with a stack — no parsing or precedence rules needed.

QUESTION 38 OF 75

Medium

Daily Temperatures

Pattern: Monotonic Stack

WHY THIS QUESTION MATTERS

THE textbook monotonic stack question — answers 'next greater element' in one pass.

Problem Statement

Given an array of daily temperatures, return an array answer where answer[i] is the number of days you'd have to wait after day i to get a warmer temperature. If none exists, answer[i] = 0.

Example

`Input: temperatures = [73,74,75,71,69,72,76,73] → Output: [1,1,4,2,1,1,0,0]`

Constraints

1 ≤ temperatures.length ≤ 10^5 · 30 ≤ temperatures[i] ≤ 100

Brute-Force Approach

For each day, scan forward until a warmer day is found: O(n²).

Optimal Approach

Maintain a stack of indices with decreasing temperatures. For each new day, while the current temperature is warmer than the temperature at the stack's top index, pop it and record the day-difference as the answer for that popped index; then push the current index.

Step-by-Step Intuition

A day only gets 'resolved' (finds its next warmer day) once a warmer day actually appears — keep unresolved days on a stack in decreasing order, and resolve as many as possible the instant a warmer day shows up.

Dry Run

`temps=[73,74,75,71,69,72,76,73]. i=0(73): stack=[0]. i=1(74)>73 → pop 0, answer[0]=1-0=1, stack=[1]. i=2(75)>74 → pop1, answer[1]=2-1=1, stack=[2]. i=3(71): stack=[2,3]. i=4(69): stack=[2,3,4]. i=5(72)>69,>71 → pop4(answer[4]=1), pop3(answer[3]=2), stack=[2,5]. i=6(76)>72,>75 → pop5(answer[5]=1), pop2(answer[2]=4), stack=[6]. i=7(73): stack=[6,7]. Remaining unresolved get 0. Final=[1,1,4,2,1,1,0,0].`

Time Complexity

O(n) — each index is pushed and popped from the stack at most once.

Space Complexity

O(n) for the stack.

Common Mistakes

Using the brute-force nested loop when O(n) is clearly expected; storing temperatures instead of indices on the stack (you need the index to compute the day-difference).

Interview Follow-up Questions

- How would you solve 'Next Greater Element' for a circular array?
- Can you adapt this pattern to find the previous smaller element instead?

Clean Python Solution

```python
def daily_temperatures(temperatures):
    n = len(temperatures)
    answer = [0] * n
    stack = []
    for i, temp in enumerate(temperatures):
        while stack and temperatures[stack[-1]] < temp:
            j = stack.pop()
            answer[j] = i - j
        stack.append(i)
    return answer
```

KEY TAKEAWAY

'Next greater/smaller element' problems are solved in one pass with a monotonic stack of indices.

QUESTION 39 OF 75

Hard

Largest Rectangle in Histogram

Pattern: Monotonic Stack

WHY THIS QUESTION MATTERS

The hardest common monotonic-stack question — a strong signal of stack mastery in interviews.

Problem Statement

Given an array of integers heights representing a histogram's bar heights (width 1 each), find the area of the largest rectangle that fits entirely within the histogram.

Example

```
Input: heights = [2,1,5,6,2,3] →
Output: 10
```

Constraints

1 ≤ heights.length ≤ 10^5 · 0 ≤ heights[i] ≤ 10^4

Brute-Force Approach

For every pair of bars, find the shortest bar between them and compute the area: O(n²), or for each bar expand outward while height stays sufficient: also O(n²) worst case.

Optimal Approach

Maintain a monotonic increasing stack of bar indices. When a shorter bar is encountered, pop taller bars off the stack, computing the area they could form using the current index as the right boundary and the new stack top as the left boundary.

Step-by-Step Intuition

A bar's maximal rectangle extends left and right until it hits a SHORTER bar. A monotonic stack naturally identifies, for each bar being popped, exactly where the nearest shorter bar on both sides is — giving the width directly.

> **Dry Run**
>
> `heights=[2,1,5,6,2,3]. i=0(2): stack=[0]. i=1(1)<2 → pop0, width=i-(-1)=1(no left bound so -1), area=2*1=2. stack=[1]. i=2(5): stack=[1,2]. i=3(6): stack=[1,2,3]. i=4(2)<6 → pop3, width=4-2-1=1,area=6. <5 → pop2,width=4-1-1=2,area=10 (max so far). stack=[1,4]. i=5(3): stack=[1,4,5]. End: pop remaining — 5(h=3,width=6-4-1=1,area=3), 4(h=2,width=6-1-1=4,area=8), 1(h=1,width=6-(-1)-1=6,area=6). Max area overall = 10.`

Time Complexity

O(n) — each bar pushed and popped once.

Space Complexity

O(n) for the stack.

Common Mistakes

Forgetting to process the remaining stack after the main loop (bars that never got popped during the scan); miscalculating width when the stack becomes empty (no left boundary — use -1 as a sentinel).

Interview Follow-up Questions

• How would you extend this to find the Maximal Rectangle in a binary matrix (uses this as a subroutine per row)?

• Can you solve this with a divide-and-conquer approach instead — what's its complexity?

Clean Python Solution

```
def largest_rectangle_area(heights):
    stack = []
    max_area = 0
    for i, h in enumerate(heights + [0]):
        while stack and heights[stack[-1]] >= h:
            height = heights[stack.pop()]
            width = i if not stack else i - stack[-1] - 1
            max_area = max(max_area, height * width)
        stack.append(i)
    return max_area
```

KEY TAKEAWAY

A monotonic stack finds, for each bar, exactly how far it can extend before hitting a shorter neighbor — in one pass.

CATEGORY 8 OF 14

Trees & BST

Recursive DFS and level-order BFS traversal patterns, plus BST-specific ordering properties.

7 questions in this section

QUESTION 40 OF 75

Easy

Invert Binary Tree

Pattern: DFS (recursive)

WHY THIS QUESTION MATTERS

The simplest recursive tree problem — establishes the 'recurse on children, combine results' template.

Problem Statement

Given the root of a binary tree, invert it (swap every left and right child) and return the root.

Example

```
Input: root = [4,2,7,1,3,6,9] →
Output: [4,7,2,9,6,3,1]
```

Constraints

0 ≤ number of nodes ≤ 100

Brute-Force Approach

N/A — the recursive solution IS the natural/optimal approach here.

Optimal Approach

Recursively invert the left and right subtrees, then swap them at the current node.

Step-by-Step Intuition

Inverting a tree means every node's children get swapped, at every level — which is naturally expressed as 'invert my children, then swap them' applied recursively down to the leaves.

Dry Run

root=4, left=2, right=7. invert(2)→swaps 2's children(1,3)→3 becomes left,1 becomes right. invert(7)→swaps(6,9)→9 left,6 right. Then swap root's children: root.left=invertedRight(7-tree), root.right=invertedLeft(2-tree). Final tree mirrored.

Time Complexity

O(n) — visits every node once.

Space Complexity

O(h) for the recursion stack, h = tree height (O(log n) balanced, O(n) worst case).

Common Mistakes

Swapping children before recursing into the ALREADY-swapped subtrees (order doesn't actually matter here since recursion is on original left/right references, but it's a common source of confusion); forgetting the base case (None node).

Interview Follow-up Questions

• Can you write this iteratively using a BFS queue instead of recursion?

• How would you verify two trees are mirror images of each other without modifying either?

Clean Python Solution

```
class TreeNode:
    def __init__(self, val=0, left=None, right=None):
        self.val = val
        self.left = left
        self.right = right

def invert_tree(root):
    if not root:
        return None
    root.left, root.right = invert_tree(root.right), invert_tree(root.left)
    return root
```

KEY TAKEAWAY

Most tree transformations are 'recurse on children, then combine' — trust the recursion to handle subtrees.

QUESTION 41 OF 75

Easy

Maximum Depth of Binary Tree

Pattern: DFS (recursive)

WHY THIS QUESTION MATTERS

Builds the 'combine children's results' recursion template used throughout tree DP.

Problem Statement

Given the root of a binary tree, return its maximum depth (the number of nodes along the longest path from root to the farthest leaf).

Example

```
Input: root = [3,9,20,null,null,15,7]
→ Output: 3
```

Constraints

0 ≤ number of nodes ≤ 10^4

Brute-Force Approach

N/A — recursion is the natural approach; an iterative BFS level-count is an equally valid alternative.

Optimal Approach

Recursively compute the depth of the left and right subtrees, and return 1 + max(leftDepth, rightDepth).

Step-by-Step Intuition

The depth of a tree rooted at a node is always one more than the deeper of its two children's depths — a direct recursive definition.

> **Dry Run**
> `root=3(children 9,20). depth(9)=1(leaf).`
> `depth(20)=1+max(depth(15),depth(7))=1+max(1,1)=2. depth(3)=1+max(1,2)=3.`

Time Complexity
`O(n) — visits every node once.`

Space Complexity
`O(h) recursion stack.`

Common Mistakes

Off-by-one (counting edges instead of nodes, or vice versa — clarify the definition with the interviewer); forgetting the base case for an empty tree (depth 0).

Interview Follow-up Questions

- Can you solve this iteratively with BFS, counting levels?
- How would you find the minimum depth (nearest leaf), and why is it trickier?

Clean Python Solution

```
def max_depth(root):
    if not root:
        return 0
    return 1 + max(max_depth(root.left), max_depth(root.right))
```

> **KEY TAKEAWAY**
> *Tree depth/height problems recurse to the leaves and combine with a simple '1 + max/min of children'.*

QUESTION 42 OF 75

Medium

Diameter of Binary Tree

Pattern: DFS (post-order accumulate global)

> WHY THIS QUESTION MATTERS
> Teaches the 'compute a global answer while returning a different local value' recursion pattern — very common in tree DP.

Problem Statement

Given the root of a binary tree, return the length (in edges) of the longest path between any two nodes in the tree. This path may or may not pass through the root.

Example
`Input: root = [1,2,3,4,5] → Output: 3`
`(path 4-2-1-3 or 5-2-1-3)`

Constraints
1 ≤ number of nodes ≤ 10^4

Brute-Force Approach

For every node, compute the height of its left and right subtrees and track the max sum: O(n²) if height is recomputed from scratch at every node.

Optimal Approach

Single DFS pass: at each node, compute left height and right height recursively; update a global/nonlocal 'best diameter' with leftHeight + rightHeight (the longest path THROUGH this node); return 1 + max(leftHeight, rightHeight) as this node's own height to the caller.

Step-by-Step Intuition

The height computation (needed anyway for recursion) is exactly what's needed at every node to also check 'is the best path through me the new global best' — do both in the same single pass instead of recomputing heights per node.

> **Dry Run**
> `root=1(left=2,right=3), 2 has children 4,5. At leaf 4: height=1. At leaf5: height=1. At node2: leftH=1,rightH=1, diameter candidate=1+1=2, returns height=1+max(1,1)=2. At leaf3: height=1. At root1: leftH=2(from node2),rightH=1(from node3), diameter candidate=2+1=3 → new global best=3.`

Time Complexity
`O(n) — one pass, unlike the O(n²) naive recomputation.`

Space Complexity
`O(h) recursion stack.`

Common Mistakes

Recomputing height as a separate top-level call per node (leads to O(n²)); confusing 'diameter in nodes' vs 'diameter in edges' (this problem counts edges).

Interview Follow-up Questions

• How would you find the diameter of a general (non-binary) tree?

• Can you also return the two actual endpoint nodes of the diameter path?

Clean Python Solution

```
def diameter_of_binary_tree(root):
    best = [0]
    def height(node):
        if not node:
            return 0
        left = height(node.left)
        right = height(node.right)
        best[0] = max(best[0], left + right)
        return 1 + max(left, right)
    height(root)
    return best[0]
```

KEY TAKEAWAY

When a recursive helper already computes what you need locally, piggyback a global best-tracker on the same pass instead of a second traversal.

QUESTION 43 OF 75

Medium

Binary Tree Level Order Traversal

Pattern: BFS

WHY THIS QUESTION MATTERS

The essential BFS-on-trees question — the template for any 'process level by level' requirement.

Problem Statement

Given the root of a binary tree, return the level order traversal of its nodes' values (i.e., from left to right, level by level), as a list of lists (one list per level).

Example

```
Input: root = [3,9,20,null,null,15,7]
→ Output: [[3],[9,20],[15,7]]
```

Constraints

0 ≤ number of nodes ≤ 2000

Brute-Force Approach

DFS while tracking depth and appending to the appropriate level's list — also valid, but BFS is the natural fit.

Optimal Approach

Use a queue. Process one full level at a time: record the current queue size, pop that many nodes, collect their values, and push their children for the next level.

Step-by-Step Intuition

A queue processes nodes in the order they were discovered (FIFO), which naturally corresponds to level-by-level order — snapshotting the queue's size before pushing children isolates exactly one level per iteration.

Dry Run

`queue=[3]. Level size=1: pop 3, record [3], push 9,20. queue=[9,20]. Level size=2: pop 9,20, record [9,20], push 20's children 15,7 (9 has none). queue=[15,7]. Level size=2: pop 15,7, record [15,7], no children. Result=[[3],[9,20],[15,7]].`

Time Complexity

`O(n) — every node enqueued and dequeued once.`

Space Complexity

`O(n) for the queue, worst case (a wide tree's last level).`

Common Mistakes

Not snapshotting the queue's length before the inner loop (mixes levels together); using DFS without tracking depth (produces the wrong grouping if not handled carefully).

Interview Follow-up Questions

- How would you return the traversal bottom-up (deepest level first)?
- How would you do a zig-zag level order traversal (alternating left-to-right and right-to-left)?

Clean Python Solution

```
from collections import deque
def level_order(root):
    if not root:
        return []
    result = []
    queue = deque([root])
    while queue:
        level = []
        for _ in range(len(queue)):
            node = queue.popleft()
            level.append(node.val)
            if node.left:
                queue.append(node.left)
            if node.right:
                queue.append(node.right)
        result.append(level)
    return result
```

KEY TAKEAWAY

Snapshot the queue's size before the inner loop to process exactly one BFS level at a time.

QUESTION 44 OF 75

Medium

Validate Binary Search Tree

Pattern: DFS (range bounds)

WHY THIS QUESTION MATTERS

Teaches the 'pass down a valid range' recursion technique — essential for any BST-property problem.

Problem Statement

Given the root of a binary tree, determine if it is a valid binary search tree (every node's value is strictly greater than all values in its left subtree, and strictly less than all values in its right subtree).

Example

```
Input: root = [5,1,4,null,null,3,6] →
Output: false  (4 is a right child of
5, but 4 < 5)
```

Constraints

1 ≤ number of nodes ≤ 10^4 · -2^31 ≤ Node.val ≤ 2^31-1

Brute-Force Approach

Just check node.left.val < node.val < node.right.val locally at every node — WRONG, because it misses violations from grandchildren further down (a node deep in the left subtree could still exceed the root).

Optimal Approach

Recursively pass down a valid (low, high) range for each node. A node must fall strictly within its allowed range; recurse left with an updated upper bound (current value) and right with an updated lower bound (current value).

Step-by-Step Intuition

A BST constraint isn't just 'compare to your parent' — it's 'compare to every ancestor that bounds you'. Carrying the accumulated (low, high) bounds down the recursion enforces the FULL ancestor constraint at every node, not just the immediate parent.

> **Dry Run**
> `root=5(range -inf,inf) → valid. left=1(range -inf,5) → valid. right=4(range 5,inf) → 4 is NOT > 5 → invalid → return False immediately.`

Time Complexity
`O(n) — visits every node once.`

Space Complexity
`O(h) recursion stack.`

Common Mistakes

Only checking a node against its immediate parent/children (the classic wrong-but-common mistake); not handling duplicate values correctly (a BST here requires STRICT inequality).

Interview Follow-up Questions

- How would you validate a BST with an in-order traversal check instead of range-passing?
- How would this change if the BST allowed duplicate values on one side?

Clean Python Solution

```
def is_valid_bst(root, low=float('-inf'), high=float('inf')):
    if not root:
        return True
    if not (low < root.val < high):
        return False
    return (is_valid_bst(root.left, low, root.val) and
            is_valid_bst(root.right, root.val, high))
```

KEY TAKEAWAY

BST validity depends on ALL ancestors, not just the parent — pass the accumulated valid range down the recursion.

QUESTION 45 OF 75

Medium

Lowest Common Ancestor of a BST

Pattern: BST Property Traversal

WHY THIS QUESTION MATTERS

Shows how BST ordering lets you find an ancestor in O(h) without searching the whole tree.

Problem Statement

Given a binary search tree and two nodes p and q, find their lowest common ancestor (the deepest node that has both p and q as descendants).

Example

Constraints

Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8 → Output: 6

2 ≤ number of nodes ≤ 10^5 · all values unique · p and q both exist in the tree

Brute-Force Approach

Find the path from root to p and root to q separately (O(h) each), then compare the paths to find where they diverge: O(h) time, O(h) space for storing paths.

Optimal Approach

Starting at root, if both p and q are smaller than the current node, go left; if both are larger, go right; otherwise (they split, or one equals current), the current node IS the LCA.

Step-by-Step Intuition

The BST property tells you directionally where p and q live relative to any node — the LCA is exactly the first node where p and q 'part ways' (one goes left, one goes right, or the node itself is one of them).

> **Dry Run**
>
> root=6, p=2, q=8. Both 2<6 and 8>6? No, they split (2<6, 8>6) → 6 is already the LCA. Return 6 immediately.

Time Complexity

O(h) — h = tree height, no need to explore both branches.

Space Complexity

O(1) iterative, O(h) if recursive.

Common Mistakes

Using the general binary tree LCA algorithm (which is O(n) and ignores the BST property entirely — correct but not optimal here); not handling the case where p or q IS the current node.

Interview Follow-up Questions

- How would you solve LCA for a general (non-BST) binary tree?
- How would you handle LCA queries for more than two nodes at once?

Clean Python Solution

```python
def lowest_common_ancestor(root, p, q):
    node = root
    while node:
        if p.val < node.val and q.val < node.val:
            node = node.left
        elif p.val > node.val and q.val > node.val:
            node = node.right
        else:
            return node
    return None
```

KEY TAKEAWAY

In a BST, the LCA is the first node where the two targets' values 'split' to opposite sides.

QUESTION 46 OF 75

Medium

Kth Smallest Element in a BST

Pattern: In-order Traversal

WHY THIS QUESTION MATTERS

Shows that an in-order traversal of a BST visits nodes in sorted order — a foundational BST fact.

Problem Statement

Given the root of a binary search tree and an integer k, return the kth smallest value among all node values in the tree.

Example

```
Input: root = [3,1,4,null,2], k = 1 →
Output: 1
```

Constraints

1 ≤ number of nodes ≤ 10^4 · 1 ≤ k ≤ number of nodes

Brute-Force Approach

Do a full in-order traversal collecting all values into a list, then return list[k-1]: O(n) time, O(n) space.

Optimal Approach

Do an in-order traversal (left, node, right) but stop early as soon as you've visited the kth node, using an explicit stack (iterative) to avoid visiting the rest of the tree.

Step-by-Step Intuition

In-order traversal of a BST always visits values in strictly ascending order — so the kth value visited IS the kth smallest, and you can stop the moment you reach it instead of collecting everything.

Dry Run

`root=3(left=1(right=2),right=4), k=1. Iterative in-order using a stack: push all left children of 3 → push 3, push1. Pop1 (no left) — that's the 1st node visited — k=1 reached → return 1.`

Time Complexity

`O(h + k) — only traverses as far left as needed, then k steps.`

Space Complexity

`O(h) for the stack.`

Common Mistakes

Collecting the entire tree into a list when early-stopping is expected as a follow-up; forgetting in-order visits LEFT, then node, then RIGHT (not node-first).

Interview Follow-up Questions

• How would you handle this efficiently if the BST is modified frequently (insertions/deletions) with many kth-smallest queries?

• Can you augment each node with a subtree-size count to answer this in O(h) without any traversal?

Clean Python Solution

```
def kth_smallest(root, k):
    stack = []
    node = root
    count = 0
    while stack or node:
        while node:
            stack.append(node)
            node = node.left
        node = stack.pop()
        count += 1
        if count == k:
            return node.val
        node = node.right
    return -1
```

KEY TAKEAWAY

In-order traversal of a BST yields values in sorted order — use it directly instead of sorting a collected list.

CATEGORY 9 OF 14

Heap / Priority Queue

When you repeatedly need the min/max element, a heap beats sorting on every operation.

4 questions in this section

QUESTION 47 OF 75

Medium

Kth Largest Element in an Array

Pattern: Heap (fixed-size min-heap)

WHY THIS QUESTION MATTERS

The essential 'maintain a fixed-size heap' pattern used for any top-K / kth-order-statistic problem.

Problem Statement

Given an integer array nums and an integer k, return the kth largest element in the array (the kth largest in sorted order, not the kth distinct element).

Example

```
Input: nums = [3,2,1,5,6,4], k = 2 →
Output: 5
```

Constraints

1 ≤ k ≤ nums.length ≤ 10^5

Brute-Force Approach

Sort the array descending and index k-1: O(n log n).

Optimal Approach

Maintain a min-heap of size k. Push every element; whenever the heap exceeds size k, pop the smallest. At the end, the heap's smallest element (its root) is the kth largest overall.

Step-by-Step Intuition

You only ever need to remember the k LARGEST elements seen so far; a size-k min-heap automatically discards anything that can't be in the top k, and its root is always the weakest of the current top-k — exactly the kth largest.

Dry Run

```
nums=[3,2,1,5,6,4], k=2. push3→[3]. push2→[2,3]. push1→ size3>2 → pop
min(1)→[2,3]. push5→[2,3,5]>2→pop2→[3,5]. push6→[3,5,6]>2→pop3→[5,6].
push4→[4,5,6]>2→pop4→[5,6]. Heap root(min)=5 → answer=5.
```

Time Complexity

O(n log k) — much better than O(n log n) full sort when k << n.

Space Complexity

O(k) for the heap.

Common Mistakes

Using a max-heap of the WHOLE array (works but wastes memory and doesn't get the O(log k) benefit); confusing 'kth largest' with 'k largest elements' (different return types).

Interview Follow-up Questions

• Can you solve this in O(n) average time using Quickselect?

• How would you maintain the kth largest element in a data stream that keeps growing?

Clean Python Solution

```
import heapq
def find_kth_largest(nums, k):
    heap = []
    for num in nums:
        heapq.heappush(heap, num)
        if len(heap) > k:
            heapq.heappop(heap)
    return heap[0]
```

KEY TAKEAWAY

A fixed-size k min-heap is the standard way to track 'the top k so far' in O(log k) per update.

QUESTION 48 OF 75

Medium

K Closest Points to Origin

Pattern: Heap (fixed-size max-heap by distance)

WHY THIS QUESTION MATTERS

Reinforces the fixed-size heap pattern with a custom comparison key (distance instead of raw value).

Problem Statement

Given an array of points on the X-Y plane and an integer k, return the k closest points to the origin (0,0).

Example

`Input: points = [[1,3],[-2,2]], k = 1`
`→ Output: [[-2,2]]`

Constraints

1 ≤ k ≤ points.length ≤ 10^4 · -10^4 ≤ coordinates ≤ 10^4

Brute-Force Approach

Compute all distances, sort by distance, take the first k: O(n log n).

Optimal Approach

Maintain a max-heap of size k keyed by squared distance (avoid sqrt for speed/precision). Push each point; if the heap exceeds size k, pop the farthest. The heap holds the k closest at the end.

Step-by-Step Intuition

Symmetric to Kth Largest Element, but this time you want to KEEP the k smallest distances, so you evict the largest whenever the heap overflows — hence a max-heap (Python's heapq is a min-heap, so negate the distance).

> **Dry Run**

points=[[1,3],[-2,2]], k=1. dist([1,3])=1+9=10. dist([-2,2])=4+4=8. Push (-10, [1,3]) → heap=[(-10,...)]. Push (-8,[-2,2]) → size2>1 → pop largest-negated i.e. smallest actual value which is -10(dist10, the farthest) → remains [-8, [-2,2]]. Result=[[-2,2]].

Time Complexity
O(n log k).

Space Complexity
O(k) for the heap.

Common Mistakes

Computing actual Euclidean distance with sqrt (unnecessary — squared distance preserves ordering and avoids floating point); using a min-heap without negating distances (evicts the wrong element).

Interview Follow-up Questions

• How would you solve this with Quickselect for O(n) average time?

• What if points arrived as a continuous stream and you needed the k closest at any moment?

Clean Python Solution

```
import heapq
def k_closest(points, k):
    heap = []
    for x, y in points:
        dist = x * x + y * y
        heapq.heappush(heap, (-dist, x, y))
        if len(heap) > k:
            heapq.heappop(heap)
    return [[x, y] for _, x, y in heap]
```

KEY TAKEAWAY

When you need the k SMALLEST by some key, use a max-heap of size k (or negate the key with Python's min-heap) and evict the largest.

QUESTION 49 OF 75

Hard

Merge K Sorted Lists

Pattern: Heap (k-way merge)

WHY THIS QUESTION MATTERS

The classic k-way merge problem — a heap generalizes the two-list merge to many lists efficiently.

Problem Statement

Given an array of k linked lists, each sorted in ascending order, merge all the lists into one sorted linked list and return it.

Example
Input: lists = [[1,4,5],[1,3,4],[2,6]]
→ Output: [1,1,2,3,4,4,5,6]

Constraints
0 ≤ k ≤ 10^4 · 0 ≤ total nodes ≤ 10^4

Brute-Force Approach

Merge lists two at a time using the Merge Two Sorted Lists technique repeatedly: O(k·n) where n is total nodes across a naive sequential merge; or collect all values, sort, rebuild: O(n log n).

Optimal Approach

Push the head of every list into a min-heap keyed by value. Repeatedly pop the smallest, append it to the result, and push that node's next (if it exists) back into the heap.

Step-by-Step Intuition

At any moment, the next smallest overall value MUST be the head of one of the k lists (since each list is individually sorted) — a heap gives O(log k) access to whichever list currently has the smallest head.

> **Dry Run**
>
> `lists=[[1,4,5],[1,3,4],[2,6]]. heap=[(1,L0),(1,L1),(2,L2)]. Pop (1,L0) → append 1, push L0.next=4 → heap has (1,L1),(2,L2),(4,L0). Pop (1,L1) → append 1, push L1.next=3 → heap has (2,L2),(3,L1),(4,L0). Pop(2,L2)→append2, push6 → continue merging until all exhausted → [1,1,2,3,4,4,5,6].`

Time Complexity

`O(n log k) — n total nodes, each heap operation O(log k).`

Space Complexity

`O(k) for the heap.`

Common Mistakes

Merging lists pairwise sequentially (correct but O(k·n), slower than the heap approach for large k); forgetting Python's heapq needs a tiebreaker when node values are equal (compare a counter or list-index alongside the value since ListNode isn't directly comparable).

Interview Follow-up Questions

- Can you solve this with divide-and-conquer pairwise merging instead — what's its complexity?
- How would this scale if k could be extremely large (e.g. 10^6 lists)?

Clean Python Solution

```
import heapq
def merge_k_lists(lists):
    heap = []
    for i, node in enumerate(lists):
        if node:
            heapq.heappush(heap, (node.val, i, node))
    dummy = ListNode()
    tail = dummy
    while heap:
        val, i, node = heapq.heappop(heap)
        tail.next = node
        tail = tail.next
        if node.next:
            heapq.heappush(heap, (node.next.val, i, node.next))
    return dummy.next
```

> **KEY TAKEAWAY**
>
> *A heap of 'current heads' generalizes two-way merging to k-way merging in O(n log k).*

QUESTION 50 OF 75

Hard

Find Median from Data Stream

Pattern: Two Heaps

WHY THIS QUESTION MATTERS

The canonical Two Heaps pattern — balancing a max-heap and min-heap to answer running-median queries in O(log n).

Problem Statement

Design a data structure that supports adding integers from a stream one at a time and finding the median of all elements added so far, at any point.

Example

`addNum(1), addNum(2), findMedian() → 1.5, addNum(3), findMedian() → 2`

Constraints

-10^5 ≤ num ≤ 10^5 · up to 5×10^4 calls to addNum and findMedian combined

Brute-Force Approach

Keep a sorted list and insert each new number in its correct position (or sort from scratch each time): O(n) per insertion.

Optimal Approach

Maintain two heaps: a max-heap for the smaller half of numbers, and a min-heap for the larger half, kept balanced in size (differing by at most 1). The median is the top of the larger heap, or the average of both tops if they're equal size.

Step-by-Step Intuition

Splitting the data stream into 'smaller half' and 'larger half' lets each heap answer 'what's the boundary value' in O(1) at its root — the median always sits right at that boundary.

Dry Run

`addNum(1): small=[-1](max-heap negated), large=[]. addNum(2): push2 to large first, then rebalance: small=[-1], large=[2]. Sizes equal(1,1) → median=avg(1,2)=1.5. addNum(3): push to large or small based on comparison, rebalance so small has one more: small=[-2,-1](i.e. holds1,2), large=[3]. Median = top of larger-sized heap = small's top = 2.`

Time Complexity

O(log n) per addNum, O(1) per findMedian.

Space Complexity

O(n) to store all elements across both heaps.

Common Mistakes

Letting the two heaps drift out of balance (must rebalance after every insertion); using a single sorted structure with O(n) insertion when O(log n) is expected.

Interview Follow-up Questions

- How would you handle this if numbers can be removed from the stream as well?
- What if the data stream is known to follow a specific distribution — can you exploit that?

Clean Python Solution

```
import heapq
class MedianFinder:
    def __init__(self):
        self.small = []  # max-heap (negated)
        self.large = []  # min-heap

    def add_num(self, num):
        heapq.heappush(self.small, -num)
        heapq.heappush(self.large, -heapq.heappop(self.small))
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def find_median(self):
        if len(self.small) > len(self.large):
            return -self.small[0]
        return (-self.small[0] + self.large[0]) / 2.0
```

KEY TAKEAWAY

Splitting a stream into a max-heap (lower half) and min-heap (upper half) answers running-median queries in O(log n).

CATEGORY 10 OF 14

Backtracking

Systematic brute force with pruning — build a solution incrementally, undo, and try again.

5 questions in this section

QUESTION 51 OF 75

Medium

Subsets

Pattern: Backtracking (include/exclude)

WHY THIS QUESTION MATTERS

The simplest backtracking template — include-or-exclude each element, the base of all subset/combination generation.

Problem Statement

Given an integer array nums of unique elements, return all possible subsets (the power set). The solution set must not contain duplicate subsets.

Example

`Input: nums = [1,2,3] → Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]`

Constraints

1 ≤ nums.length ≤ 10 · all elements distinct

Brute-Force Approach

Generate all 2^n bitmask combinations iteratively and build subsets from each: also O(2^n), essentially equivalent in cost to backtracking.

Optimal Approach

Backtrack: at each index, recurse once WITH the current element included in the running subset, and once WITHOUT it; record the current subset at every recursive call (every path is a valid subset).

Step-by-Step Intuition

Every subset corresponds to a sequence of n binary decisions ('include element i or not') — backtracking simply explores both branches of that decision tree, undoing each choice before trying the other.

Dry Run

`nums=[1,2,3]. Start path=[]. Record []. Include1→path=[1], record. Include2→path=[1,2],record. Include3→path=[1,2,3],record. Backtrack3,exclude3→already recorded[1,2]. Backtrack2, exclude2, include3→path=[1,3],record. Continue exhaustively — 8 total subsets recorded.`

Time Complexity

Space Complexity

O(n · 2^n) — 2^n subsets, each up to O(n) to copy.

O(n) recursion depth, O(n·2^n) for the output.

Common Mistakes

Forgetting to record the subset at EVERY node of the recursion (not just at leaves); appending a reference to the mutable path list instead of a copy (all recorded subsets end up identical/empty).

Interview Follow-up Questions

• How would this change if nums contained duplicates (Subsets II)?

• Can you generate subsets iteratively using bitmasks instead of recursion?

Clean Python Solution

```
def subsets(nums):
    result = []
    path = []
    def backtrack(start):
        result.append(path[:])
        for i in range(start, len(nums)):
            path.append(nums[i])
            backtrack(i + 1)
            path.pop()
    backtrack(0)
    return result
```

KEY TAKEAWAY

Backtracking's core loop is: choose, recurse, undo (pop) — record a valid state at every step, not just at the end.

QUESTION 52 OF 75

Medium

Permutations

Pattern: Backtracking (choose/unchoose with used-set)

WHY THIS QUESTION MATTERS

Extends backtracking to track 'which elements are already used' — the base pattern for any ordering-sensitive generation.

Problem Statement

Given an array nums of distinct integers, return all possible permutations, in any order.

Example

Input: nums = [1,2,3] → Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]

Constraints

1 ≤ nums.length ≤ 6 · all elements distinct

Brute-Force Approach

N/A — generating all n! permutations inherently requires exploring the full decision tree; backtracking IS the standard approach.

Optimal Approach

Backtrack by building a path one element at a time; at each step, try every not-yet-used element, mark it used, recurse, then unmark it (undo) before trying the next.

Step-by-Step Intuition

A permutation is an ordering, so at each position you choose from whatever elements remain — track 'used' elements explicitly, and undo the choice after exploring so the next branch starts clean.

> **Dry Run**
>
> `nums=[1,2,3]. path=[],used={}. Choose1→path=[1]. Choose2→path=[1,2]. Choose3→path=[1,2,3],record,backtrack. Undo3→path=[1,2],no more choices,backtrack. Undo2→path=[1]. Choose3→path=[1,3]. Choose2→path=[1,3,2],record. Continue exhaustively for all 6 permutations.`

Time Complexity

`O(n · n!)` — `n!` permutations, each `O(n)` to build/copy.

Space Complexity

`O(n)` recursion depth plus `O(n)` for the used-set.

Common Mistakes

Forgetting to 'unchoose' (remove from used-set and pop from path) before trying the next branch — corrupts subsequent branches; not copying the path when recording the final result.

Interview Follow-up Questions

- How would you handle duplicate elements in nums (Permutations II)?
- Can you generate the next lexicographic permutation without generating all of them (Next Permutation)?

Clean Python Solution

```python
def permute(nums):
    result = []
    path = []
    used = [False] * len(nums)
    def backtrack():
        if len(path) == len(nums):
            result.append(path[:])
            return
        for i, num in enumerate(nums):
            if used[i]:
                continue
            used[i] = True
            path.append(num)
            backtrack()
            path.pop()
            used[i] = False
    backtrack()
    return result
```

KEY TAKEAWAY

Permutation backtracking needs an explicit 'used' tracker, since order matters and every element must appear exactly once per path.

QUESTION 53 OF 75

Medium

Combination Sum

Pattern: Backtracking (unbounded choice + pruning)

WHY THIS QUESTION MATTERS

Introduces pruning (cutting a branch early once it can't succeed) — essential for keeping backtracking fast.

Problem Statement

Given an array of distinct positive integers candidates and a target, return all unique combinations where the chosen numbers sum to target. The same number may be chosen an unlimited number of times.

Example

```
Input: candidates = [2,3,6,7], target = 7 → Output: [[2,2,3],[7]]
```

Constraints

2 ≤ candidates.length ≤ 30 · 1 ≤ candidates[i] ≤ 200 · all candidates distinct · 1 ≤ target ≤ 500

Brute-Force Approach

Explore every possible combination of any length without pruning: exponential blow-up, often impractically slow.

Optimal Approach

Backtrack starting from an index, trying to include candidates[i] repeatedly (allowing reuse), subtracting from a remaining target, and only recursing while remaining ≥ 0; stop (prune) once remaining < 0.

Step-by-Step Intuition

Since numbers can repeat, always pass the SAME index forward (not index+1) when reusing a number, but never go backward to earlier indices (that would create duplicate combinations in a different order). Prune immediately once the running sum overshoots target.

Dry Run

```
candidates=[2,3,6,7], target=7.
Choose2(remaining5)→choose2(remaining3)→choose2(remaining1)→choose2(remaining-1) prune. Backtrack, try3(remaining-2 from1) prune... eventually path[2,2,3] sums to7 → record. Later path starting fresh with7(remaining0)→record[7].
```

Time Complexity

```
Exponential in the worst case, bounded by the pruning — roughly O(2^target / candidates) practically much faster due to early cutoffs.
```

Space Complexity

```
O(target/min(candidates)) recursion depth.
```

Common Mistakes

Passing i+1 instead of i when recursing (would forbid reusing the same number, giving the wrong answer for THIS variant); not pruning early (remaining < 0 check), leading to wasted exploration.

Interview Follow-up Questions

• How would this change if each number could only be used once (Combination Sum II, with duplicates in the input)?

• How would you count the number of combinations instead of listing them (often solvable with DP instead)?

Clean Python Solution

```
def combination_sum(candidates, target):
    result = []
    path = []
    def backtrack(start, remaining):
        if remaining == 0:
            result.append(path[:])
            return
        if remaining < 0:
            return
        for i in range(start, len(candidates)):
            path.append(candidates[i])
            backtrack(i, remaining - candidates[i])
            path.pop()
    backtrack(0, target)
    return result
```

KEY TAKEAWAY

Prune backtracking branches the instant they can't succeed (remaining < 0) — don't wait to discover failure at the leaf.

QUESTION 54 OF 75

Medium

Word Search

Pattern: Backtracking / DFS on Grid

WHY THIS QUESTION MATTERS

Combines grid DFS with backtracking's 'mark, recurse, unmark' discipline — a very common interview format.

Problem Statement

Given an m x n grid of characters board and a string word, return true if word exists in the grid, formed by sequentially adjacent cells (horizontally or vertically neighboring), using each cell at most once.

Example

```
Input: board = [["A","B","C","E"],
["S","F","C","S"],["A","D","E","E"]],
word = "ABCCED" → Output: true
```

Constraints

$1 \le m,n \le 6 \cdot 1 \le word.length \le 15$

Brute-Force Approach

Try starting the search from every cell and explore all 4 directions naively without marking visited cells: leads to infinite loops or massively redundant exploration.

Optimal Approach

For each starting cell matching word[0], DFS in 4 directions, matching subsequent characters; temporarily mark visited cells (e.g. overwrite with a sentinel), and restore them (unmark) after backtracking out of that path.

Step-by-Step Intuition

A cell can't be reused within the SAME path, but it CAN be reused by a different starting path — so mark it visited only for the duration of the current DFS branch, then undo the mark on the way back up (classic backtracking).

> **Dry Run**
>
> board as given, word="ABCCED". Start at (0,0)='A' matches word[0]. Mark visited. Try neighbor (0,1)='B' matches word[1]. Mark visited. Try (0,2)='C' matches word[2]. Mark. Try (1,2)='C' matches word[3]. Mark. Try (2,2)='E' matches word[4]. Mark. Try (2,1)='D' matches word[5] — full word matched → return True (unmarking happens only if a branch fails, which none did here).

Time Complexity

O(m·n·4^L) worst case, where L = word length (4 directions per step, pruned heavily by character mismatches in practice).

Space Complexity

O(L) recursion depth for the current path.

Common Mistakes

Forgetting to unmark (restore) a cell after backtracking out of a failed path (breaks other candidate paths); not checking grid boundaries before indexing.

Interview Follow-up Questions

• How would you search for MULTIPLE words efficiently in the same grid (Word Search II, using a Trie)?

• How would you handle diagonal adjacency instead of just horizontal/vertical?

Clean Python Solution

```
def exist(board, word):
    rows, cols = len(board), len(board[0])
    def dfs(r, c, i):
        if i == len(word):
            return True
        if r < 0 or r >= rows or c < 0 or c >= cols or board[r][c] != word[i]:
            return False
        temp = board[r][c]
        board[r][c] = '#'
        found = (dfs(r + 1, c, i + 1) or dfs(r - 1, c, i + 1) or
                 dfs(r, c + 1, i + 1) or dfs(r, c - 1, i + 1))
        board[r][c] = temp
        return found
    for r in range(rows):
        for c in range(cols):
            if dfs(r, c, 0):
                return True
    return False
```

> **KEY TAKEAWAY**
>
> *Grid backtracking marks a cell visited for the current path only, and always restores it before returning — mark, recurse, unmark.*

QUESTION 55 OF 75

Hard

N-Queens

Pattern: Backtracking (constraint propagation)

WHY THIS QUESTION MATTERS

The definitive hard backtracking question — teaches tracking multiple simultaneous constraints (columns, diagonals) efficiently.

Problem Statement

Place n queens on an n x n chessboard so that no two queens attack each other (same row, column, or diagonal). Return all distinct solutions, each as a board configuration.

Example

```
Input: n = 4 → Output: 2 solutions
(each a 4x4 board with one queen per
row/column, no shared diagonals)
```

Constraints

$1 \le n \le 9$

Brute-Force Approach

Try placing queens in every possible cell combination and check all pairs for conflicts at the end: extremely wasteful, effectively O(n^(2n)).

Optimal Approach

Place one queen per row, backtracking column by column. Track used columns and both diagonal directions (row-col and row+col are constant along each diagonal) with sets, so conflict checks are O(1) instead of scanning the board.

Step-by-Step Intuition

Since each row can hold exactly one queen, you only need to choose a COLUMN per row — reducing the search space from all cells to just column choices, and diagonal conflicts can be checked instantly using the row±col invariant instead of scanning.

Dry Run

```
n=4. Row0: try col0 — place, mark col0,diag(0-0)=0,diag(0+0)=0. Row1: try
col0(conflict,same col)→try col1(conflict,diag 1-1=0 matches)→try col2 (no
conflicts) — place. Row2: try each col, all conflict with existing queens —
backtrack row1. Try col1 fails to progress further... eventually valid full
placements found: columns [1,3,0,2] and [2,0,3,1] — 2 solutions for n=4.
```

Time Complexity

O(n!) worst case (heavily pruned in practice by the O(1) constraint checks).

Space Complexity

O(n) for the column/diagonal tracking sets plus O(n) recursion depth.

Common Mistakes

Checking conflicts by scanning the whole board each time (O(n) per check instead of O(1) with sets — makes the solution far too slow for larger n); forgetting one of the two diagonal directions.

Interview Follow-up Questions

- Can you solve N-Queens II (count solutions only) more efficiently by skipping board reconstruction?
- How would this generalize to placing multiple different piece types with different attack patterns?

Clean Python Solution

```
def solve_n_queens(n):
    result = []
    cols, diag1, diag2 = set(), set(), set()
    board = [['.'] * n for _ in range(n)]
    def backtrack(row):
        if row == n:
            result.append([''.join(r) for r in board])
            return
        for col in range(n):
            if col in cols or (row - col) in diag1 or (row + col) in diag2:
                continue
            cols.add(col); diag1.add(row - col); diag2.add(row + col)
            board[row][col] = 'Q'
            backtrack(row + 1)
            board[row][col] = '.'
            cols.remove(col); diag1.remove(row - col); diag2.remove(row + col)
    backtrack(0)
    return result
```

KEY TAKEAWAY

Track constraints (columns, diagonals) with O(1)-checkable sets instead of rescanning the board — the difference between fast and impossibly slow backtracking.

CATEGORY 11 OF 14

Graphs

DFS, BFS, topological sort, union-find, and shortest paths on general (non-tree) structures.

7 questions in this section

QUESTION 56 OF 75

Medium

Number of Islands

Pattern: DFS/BFS on Grid

WHY THIS QUESTION MATTERS

The foundational grid-graph question — connected-component counting via DFS/BFS.

Problem Statement

Given an m x n binary grid where '1' is land and '0' is water, return the number of islands (a group of horizontally/vertically connected '1's).

Example

```
Input: grid = [["1","1","0"],["1","0","0"],["0","0","1"]] → Output: 2
```

Constraints

1 ≤ m,n ≤ 300

Brute-Force Approach

N/A — DFS/BFS flood fill is the natural and optimal approach.

Optimal Approach

Scan every cell; whenever an unvisited '1' is found, increment the island count and flood-fill (DFS or BFS) to mark every connected '1' as visited so it isn't counted again.

Step-by-Step Intuition

Each flood-fill call consumes one entire connected component, so the number of times you START a new flood-fill IS the number of islands.

Dry Run

```
grid as given. Scan (0,0)='1', unvisited → count=1, flood-fill marks (0,0),(0,1),(1,0) visited. Scan continues, (0,2)='0' skip. (2,2)='1' unvisited → count=2, flood-fill marks it. Total islands=2.
```

Time Complexity

O(m·n) — every cell visited at most once.

Space Complexity

O(m·n) worst case for the recursion/BFS queue.

Common Mistakes

Not marking cells visited (infinite loop or overcounting); forgetting to check grid boundaries before recursing into neighbors.

Interview Follow-up Questions

• How would you count islands if diagonal connections also counted?

• How would you solve this with Union-Find instead of DFS/BFS?

Clean Python Solution

```
def num_islands(grid):
    if not grid:
        return 0
    rows, cols = len(grid), len(grid[0])
    def dfs(r, c):
        if r < 0 or r >= rows or c < 0 or c >= cols or grid[r][c] != '1':
            return
        grid[r][c] = '0'
        dfs(r + 1, c); dfs(r - 1, c); dfs(r, c + 1); dfs(r, c - 1)
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                count += 1
                dfs(r, c)
    return count
```

KEY TAKEAWAY

Counting connected components on a grid: scan for unvisited starts, flood-fill each one, count the starts.

QUESTION 57 OF 75

Medium

Clone Graph

Pattern: DFS/BFS + Hash Map (visited copies)

WHY THIS QUESTION MATTERS

Teaches how to traverse a graph WHILE building a parallel structure, using a map to avoid infinite loops on cycles.

Problem Statement

Given a reference node in a connected undirected graph, return a deep copy (clone) of the graph.

Example

`Input: adjList = [[2,4],[1,3],[2,4],[1,3]]` → `Output: a structurally identical cloned graph`

Constraints

0 ≤ number of nodes ≤ 100 · no repeated edges or self-loops

Brute-Force Approach

N/A — DFS/BFS with a visited map is the standard, necessary approach given the graph may contain cycles.

Optimal Approach

DFS (or BFS) from the given node, using a hash map from original node → cloned node. Before recursing into a neighbor, check if it's already been cloned (present in the map); if so, reuse that clone instead of recursing again.

Step-by-Step Intuition

Because the graph can have cycles, a naive traversal would recurse forever — the visited map both prevents infinite recursion AND lets you correctly wire up clones that reference each other in a cycle.

> **Dry Run**
> `Start node1. map={}. Clone node1→map={1:clone1}. Visit neighbors 2,4. Clone2 not in map→clone it, map={1:c1,2:c2}, recurse into 2's neighbors(1,3): 1 already in map → reuse c1. Clone3→map adds 3:c3, recurse into 3's neighbors(2,4): 2 in map→reuse. Continue until all nodes cloned and wired.`

Time Complexity
`O(V + E) — every node and edge visited once.`

Space Complexity
`O(V) for the map and recursion stack.`

Common Mistakes

Forgetting to check the map BEFORE recursing (causes infinite recursion on any cycle); not copying the neighbor list correctly (reusing original node references instead of cloned ones).

Interview Follow-up Questions

- How would you clone a graph with weighted edges?
- Could you do this iteratively with BFS instead of DFS — what changes?

Clean Python Solution

```
class Node:
    def __init__(self, val=0, neighbors=None):
        self.val = val
        self.neighbors = neighbors or []

def clone_graph(node):
    if not node:
        return None
    visited = {}
    def dfs(n):
        if n in visited:
            return visited[n]
        copy = Node(n.val)
        visited[n] = copy
        for neighbor in n.neighbors:
            copy.neighbors.append(dfs(neighbor))
        return copy
    return dfs(node)
```

KEY TAKEAWAY

A visited map during graph traversal both prevents infinite cycles and lets you correctly wire cloned references.

QUESTION 58 OF 75

Medium

Course Schedule

Pattern: Topological Sort (cycle detection)

WHY THIS QUESTION MATTERS

The essential 'can this be done at all' topological-sort question — cycle detection in a directed graph.

Problem Statement

There are numCourses courses labeled 0 to numCourses-1. Given prerequisite pairs [a, b] meaning you must take b before a, return true if you can finish all courses (i.e., no circular dependency exists).

Example

```
Input: numCourses = 2, prerequisites = [[1,0]] → Output: true
Input: numCourses = 2, prerequisites = [[1,0],[0,1]] → Output: false
```

Constraints

1 ≤ numCourses ≤ 2000 · 0 ≤ prerequisites.length ≤ 5000

Brute-Force Approach

N/A — cycle detection inherently requires a graph traversal; there's no meaningfully simpler brute force.

Optimal Approach

Build an adjacency list and compute in-degrees (Kahn's algorithm). Start a queue with all in-degree-0 nodes; repeatedly remove a node, decrement its neighbors' in-degrees, and enqueue any that reach 0. If all nodes get processed, there's no cycle.

Step-by-Step Intuition

A course can only be taken once all its prerequisites are satisfied — in-degree-0 nodes represent 'currently takeable' courses. If a cycle exists, some subset of nodes will NEVER reach in-degree 0, and they'll be left unprocessed at the end.

Dry Run

`numCourses=2, prereqs=[[1,0]] means edge 0→1. in-degree: node0=0, node1=1. Queue=[0]. Process0, decrement node1's in-degree to0, enqueue1. Process1. Processed count=2==numCourses → True (no cycle).`

Time Complexity

`O(V + E)` — build graph + process each node/edge once.

Space Complexity

`O(V + E)` for the adjacency list and in-degree array.

Common Mistakes

Using DFS with a simple visited set (doesn't distinguish 'currently in recursion stack' from 'fully processed', which is needed to detect a cycle correctly — needs a 3-color/in-progress marker if going the DFS route); miscounting in-degrees from the (a, b) pair direction.

Interview Follow-up Questions

• How would you also return a VALID course order, not just true/false (Course Schedule II)?

• How would you detect a cycle using DFS with node coloring instead of Kahn's algorithm?

Clean Python Solution

```
from collections import deque, defaultdict
def can_finish(num_courses, prerequisites):
    graph = defaultdict(list)
    in_degree = [0] * num_courses
    for course, prereq in prerequisites:
        graph[prereq].append(course)
        in_degree[course] += 1
    queue = deque([c for c in range(num_courses) if in_degree[c] == 0])
    processed = 0
    while queue:
        node = queue.popleft()
        processed += 1
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)
    return processed == num_courses
```

KEY TAKEAWAY

Kahn's algorithm detects a cycle by checking whether every node eventually reaches in-degree 0 and gets processed.

QUESTION 59 OF 75

Medium

Course Schedule II

Pattern: Topological Sort (ordering)

WHY THIS QUESTION MATTERS

Extends cycle detection to produce an actual valid order — the real-world 'build order' use case.

Problem Statement

Given numCourses and prerequisite pairs, return an ordering of courses you could take to finish all of them. If impossible, return an empty array.

Example

`Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]] → Output: [0,1,2,3] (or [0,2,1,3])`

Constraints

1 ≤ numCourses ≤ 2000 · 0 ≤ prerequisites.length ≤ 5000

Brute-Force Approach

N/A — same as Course Schedule; a valid order can only come from a proper topological traversal.

Optimal Approach

Same Kahn's algorithm as Course Schedule, but append each processed node to a result list as it's dequeued; return that list if all nodes were processed, else return empty.

Step-by-Step Intuition

The ORDER in which Kahn's algorithm processes in-degree-0 nodes IS a valid topological order — the cycle-detection check and the ordering come for free from the same process.

> **Dry Run**
> `numCourses=4, edges: 0→1,0→2,1→3,2→3. in-degree:[0,1,1,2]. Queue=[0]. Process0→order=[0], decrement1,2 to0 → queue=[1,2]. Process1→order=[0,1], decrement3 to1. Process2→order=[0,1,2], decrement3 to0→queue=[3]. Process3→order=[0,1,2,3]. All 4 processed → valid.`

Time Complexity
`O(V + E)`.

Space Complexity
`O(V + E)`.

Common Mistakes

Forgetting to check that ALL nodes were processed before returning the order (a partial order from a graph with a cycle is invalid and must return empty); building the order from a DFS post-order without reversing it (DFS-based topological sort requires reversing the post-order stack).

Interview Follow-up Questions

• How would you return ALL valid topological orderings, not just one?

• How would you detect which specific courses are involved in a cycle, to report a clearer error?

Clean Python Solution

```
from collections import deque, defaultdict
def find_order(num_courses, prerequisites):
    graph = defaultdict(list)
    in_degree = [0] * num_courses
    for course, prereq in prerequisites:
        graph[prereq].append(course)
        in_degree[course] += 1
    queue = deque([c for c in range(num_courses) if in_degree[c] == 0])
    order = []
    while queue:
        node = queue.popleft()
        order.append(node)
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)
    return order if len(order) == num_courses else []
```

> **KEY TAKEAWAY**
> *Kahn's algorithm's processing order IS a valid topological sort — cycle detection and ordering are the same computation.*

QUESTION 60 OF 75

Medium

Pacific Atlantic Water Flow

Pattern: Multi-source DFS/BFS

WHY THIS QUESTION MATTERS

Teaches the 'reverse the flow, start from the boundary' trick and multi-source traversal — both widely reusable.

Problem Statement

Given an m x n grid of heights, water can flow from a cell to a neighboring cell with height ≤ its own. Return all cells from which water can reach BOTH the Pacific (top/left edges) and Atlantic (bottom/right edges) oceans.

Example

```
Input: heights = [[1,2,2,3,5],
[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],
[5,1,1,2,4]] → Output: cells like
[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],
[4,0]
```

Constraints

1 ≤ m,n ≤ 200

Brute-Force Approach

For every cell, run a DFS/BFS to check if it can reach BOTH oceans independently: O((m·n)²), far too slow.

Optimal Approach

Reverse the problem: start DFS/BFS from every Pacific-adjacent cell (top row + left column) moving to neighbors with height ≥ current (reverse flow), marking reachable cells; do the same from Atlantic-adjacent cells (bottom row + right column). Cells reachable from BOTH searches are the answer.

Step-by-Step Intuition

Instead of checking 'can I reach the ocean' from every cell (expensive, redundant), check 'can the ocean reach me' by flowing backward from each ocean's border — a single multi-source search per ocean covers every cell in one pass.

Dry Run

```
Start Pacific search from all top-row and left-column cells simultaneously
(multi-source BFS/DFS), flowing to neighbors with height >= current (reverse of
the real flow direction). Mark visited set pacificReachable. Do the same for
Atlantic from bottom-row/right-column → atlanticReachable. Intersect both sets
for the final answer.
```

Time Complexity

O(m·n) — two multi-source traversals, each visiting every cell at most once.

Space Complexity

O(m·n) for the two visited sets.

Common Mistakes

Running a separate traversal per cell instead of multi-source from the borders (the O((mn)²) brute-force mistake); getting the flow-reversal comparison backward (should move to neighbors with height ≥ current, since real water flows downhill).

Interview Follow-up Questions

- How would you solve this if there were more than two 'oceans' (arbitrary boundary sets)?
- Could you solve it with Union-Find instead of DFS/BFS?

Clean Python Solution

```
def pacific_atlantic(heights):
    if not heights:
        return []
    rows, cols = len(heights), len(heights[0])
    pacific, atlantic = set(), set()
    def dfs(r, c, visited, prev_height):
        if (r, c) in visited or r < 0 or r >= rows or c < 0 or c >= cols or
heights[r][c] < prev_height:
            return
        visited.add((r, c))
        for dr, dc in ((1,0),(-1,0),(0,1),(0,-1)):
            dfs(r + dr, c + dc, visited, heights[r][c])
    for c in range(cols):
        dfs(0, c, pacific, heights[0][c])
        dfs(rows - 1, c, atlantic, heights[rows - 1][c])
    for r in range(rows):
        dfs(r, 0, pacific, heights[r][0])
        dfs(r, cols - 1, atlantic, heights[r][cols - 1])
    return [list(cell) for cell in pacific & atlantic]
```

KEY TAKEAWAY

When checking reachability FROM many targets, it's often cheaper to flow backward from those targets instead of forward from every source.

QUESTION 61 OF 75

Medium

Graph Valid Tree

Pattern: Union-Find / DFS Cycle Check

WHY THIS QUESTION MATTERS

Introduces Union-Find as an alternative to DFS for connectivity and cycle checks — a core structure for many graph problems.

Problem Statement

Given n nodes labeled 0 to n-1 and a list of undirected edges, determine if these edges form a valid tree (connected, and exactly n-1 edges with no cycles).

Example

`Input: n = 5, edges = [[0,1],[0,2],[0,3],[1,4]] → Output: true`
`Input: n = 5, edges = [[0,1],[1,2],[2,3],[1,3],[1,4]] → Output: false`

Constraints

1 ≤ n ≤ 2000 · 0 ≤ edges.length ≤ 5000 · no self-loops or duplicate edges

Brute-Force Approach

Run a full DFS/BFS, checking for cycles by tracking parent pointers, then separately verify all nodes were visited (connectivity): works but a bit more bookkeeping than Union-Find.

Optimal Approach

A graph is a valid tree if and only if it has exactly n-1 edges AND is fully connected. Use Union-Find: for each edge, if both endpoints are already in the same set, a cycle exists (invalid); otherwise union them. At the end, check exactly one connected component remains.

Step-by-Step Intuition

A tree is defined by having no cycles and being fully connected — Union-Find directly detects 'these two nodes are already connected' (a cycle) in near O(1), and counting the final distinct sets confirms connectivity.

> **Dry Run**
> `n=5, edges=[[0,1],[0,2],[0,3],[1,4]]. Edge count=4=n-1 ✓. Union(0,1): different sets→merge. Union(0,2): different→merge. Union(0,3): different→merge. Union(1,4): different→merge. No cycle found, all edges processed. Final component count=1 → valid tree → True.`

Time Complexity
`O(n α(n)) — near-linear with union-find's inverse-Ackermann amortized cost.`

Space Complexity
`O(n) for the union-find parent array.`

Common Mistakes

Forgetting the edge-count check upfront (a graph can have n-1 edges but still be disconnected with a separate cycle elsewhere — both conditions are necessary); not using path compression/union by rank, degrading Union-Find's efficiency.

Interview Follow-up Questions

• How would you find the number of connected components in a general graph (not necessarily a tree check)?

• How would you detect a cycle in a DIRECTED graph instead — does Union-Find still work directly?

Clean Python Solution

```
def valid_tree(n, edges):
    if len(edges) != n - 1:
        return False
    parent = list(range(n))
    def find(x):
        while parent[x] != x:
            parent[x] = parent[parent[x]]
            x = parent[x]
        return x
    for a, b in edges:
        ra, rb = find(a), find(b)
        if ra == rb:
            return False
        parent[ra] = rb
    return True
```

KEY TAKEAWAY

A valid tree needs exactly n-1 edges AND no cycles — Union-Find checks both in near-linear time.

QUESTION 62 OF 75

Medium

Network Delay Time

Pattern: Dijkstra's Algorithm

WHY THIS QUESTION MATTERS

The standard weighted-shortest-path introduction — Dijkstra's algorithm with a heap.

Problem Statement

Given a network of n nodes with travel times as directed weighted edges times = [[u, v, w], ...], and a starting node k, return the minimum time for a signal from k to reach all n nodes. Return -1 if impossible.

Example

```
Input: times = [[2,1,1],[2,3,1],
[3,4,1]], n = 4, k = 2 → Output: 2
```

Constraints

1 ≤ n ≤ 100 · 1 ≤ times.length ≤ 6000 · 0 ≤ w ≤ 100

Brute-Force Approach

Bellman-Ford: relax all edges n-1 times: O(V·E), correct but slower than Dijkstra when there are no negative weights.

Optimal Approach

Dijkstra's algorithm with a min-heap: start at k with distance 0; repeatedly pop the closest unvisited node, relax its outgoing edges, and push improved distances. Track the max finalized distance across all nodes.

Step-by-Step Intuition

Since all weights are non-negative, once a node is popped from the min-heap with its current best distance, that distance can never be improved later — so processing nodes in increasing distance order (a heap) greedily finalizes each node's shortest path exactly once.

Dry Run

```
times=[[2,1,1],[2,3,1],[3,4,1]], n=4,k=2. dist={2:0}. heap=[(0,2)]. Pop(0,2):
relax 2→1(dist1=1),2→3(dist3=1). heap=[(1,1),(1,3)]. Pop(1,1): no outgoing
edges. Pop(1,3): relax 3→4(dist4=2). heap=[(2,4)]. Pop(2,4): no edges. Final
dist={2:0,1:1,3:1,4:2}. Max=2 → answer=2.
```

Time Complexity

O(E log V) with a binary heap.

Space Complexity

O(V + E) for the graph and distance tracking.

Common Mistakes

Using Dijkstra on a graph with NEGATIVE edge weights (invalid — Dijkstra assumes non-negative weights; Bellman-Ford is needed instead); forgetting to check that every node was actually reached before returning the max distance.

Interview Follow-up Questions

• How would this change if some edge weights could be negative?

• How would you find the actual shortest PATH, not just the time, from k to a specific node?

Clean Python Solution

```
import heapq
from collections import defaultdict
def network_delay_time(times, n, k):
    graph = defaultdict(list)
    for u, v, w in times:
        graph[u].append((v, w))
    dist = {}
    heap = [(0, k)]
    while heap:
        d, node = heapq.heappop(heap)
        if node in dist:
            continue
        dist[node] = d
        for neighbor, weight in graph[node]:
            if neighbor not in dist:
                heapq.heappush(heap, (d + weight, neighbor))
    return max(dist.values()) if len(dist) == n else -1
```

KEY TAKEAWAY

Dijkstra's greedily finalizes the shortest distance to the closest unvisited node first — correct only when all edge weights are non-negative.

CATEGORY 12 OF 14

Dynamic Programming

Break a problem into overlapping subproblems and cache the answers — 1D and 2D DP tables.

7 questions in this section

QUESTION 63 OF 75

Easy

Climbing Stairs

Pattern: 1D DP (Fibonacci-shaped)

WHY THIS QUESTION MATTERS

The gentlest possible introduction to DP — shows how overlapping subproblems appear even in a trivial-looking recursion.

Problem Statement

You are climbing a staircase with n steps. Each time you can climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example

`Input: n = 3 → Output: 3  (1+1+1, 1+2, 2+1)`

Constraints

$1 \le n \le 45$

Brute-Force Approach

Plain recursion: ways(n) = ways(n-1) + ways(n-2), recomputing the same subproblems exponentially: O(2^n).

Optimal Approach

Bottom-up DP: build an array (or just two running variables) where dp[i] = dp[i-1] + dp[i-2], starting from dp[1]=1, dp[2]=2.

Step-by-Step Intuition

The last move to reach step n was either a 1-step from n-1, or a 2-step from n-2 — so the total ways to reach n is simply the sum of ways to reach those two prior steps, exactly the Fibonacci recurrence.

Dry Run

`n=3. dp[1]=1, dp[2]=2. dp[3]=dp[2]+dp[1]=2+1=3. Answer=3.`

Time Complexity

`O(n) with DP (vs O(2^n) naive recursion).`

Space Complexity

`O(1) using two rolling variables instead of a full array.`

Common Mistakes

Using naive recursion without memoization (times out for n as small as ~40); off-by-one on the base cases (dp[1] and dp[2]).

Interview Follow-up Questions

• What if you could climb 1, 2, or 3 steps at a time — how does the recurrence change?

• Can you solve this with matrix exponentiation for O(log n) time?

Clean Python Solution

```
def climb_stairs(n):
    if n <= 2:
        return n
    prev2, prev1 = 1, 2
    for _ in range(3, n + 1):
        prev2, prev1 = prev1, prev1 + prev2
    return prev1
```

KEY TAKEAWAY

Recognizing 'the answer depends on the previous 1-2 states' is the entry point into 1D DP — replace recursion with a rolling computation.

QUESTION 64 OF 75

Medium

House Robber

Pattern: 1D DP (take/skip decision)

WHY THIS QUESTION MATTERS

The template 'take or skip' 1D DP — extremely common in interviews under many disguises.

Problem Statement

Given an array nums representing money in houses arranged in a line, find the maximum amount you can rob without robbing two adjacent houses.

Example

`Input: nums = [2,7,9,3,1] → Output: 12`
`(rob houses 0, 2, 4: 2+9+1=12)`

Constraints

1 ≤ nums.length ≤ 100 · 0 ≤ nums[i] ≤ 400

Brute-Force Approach

Try every subset of non-adjacent houses: exponential, O(2^n).

Optimal Approach

DP where dp[i] = max(dp[i-1], dp[i-2] + nums[i]) — either skip house i (keep dp[i-1]) or rob it (dp[i-2] plus its value). Collapse to two rolling variables.

Step-by-Step Intuition

At each house, you face a binary decision: rob it (and add to the best up to two houses back) or don't (keep the best up to the previous house) — always take whichever is larger.

> **Dry Run**

nums=[2,7,9,3,1]. dp[0]=2. dp[1]=max(2,7)=7. dp[2]=max(7,2+9)=11. dp[3]=max(11,7+3)=11. dp[4]=max(11,11+1)=12. Answer=12.

Time Complexity
O(n).

Space Complexity
O(1) with rolling variables (O(n) with a full DP array).

Common Mistakes

Forgetting that dp[i-2]+nums[i] competes with dp[i-1], not simply alternating houses greedily (greedy alternating fails on inputs like [2,1,1,9]); off-by-one in base cases for small n (0 or 1 house).

Interview Follow-up Questions

• How does this change if the houses are arranged in a CIRCLE (House Robber II)?

• How would you also return WHICH houses were robbed, not just the max amount?

Clean Python Solution

```
def rob(nums):
    prev2, prev1 = 0, 0
    for num in nums:
        prev2, prev1 = prev1, max(prev1, prev2 + num)
    return prev1
```

KEY TAKEAWAY

'Take or skip, with a gap constraint' problems reduce to dp[i] = max(skip, take + dp[i-2]).

QUESTION 65 OF 75

Medium

Longest Increasing Subsequence

Pattern: 1D DP (with O(n log n) optimization)

WHY THIS QUESTION MATTERS

A landmark DP problem that also teaches a non-obvious O(n log n) optimization using binary search.

Problem Statement

Given an integer array nums, return the length of the longest strictly increasing subsequence.

Example
Input: nums = [10,9,2,5,3,7,101,18] →
Output: 4 ([2,3,7,101] or [2,3,7,18])

Constraints
1 ≤ nums.length ≤ 2500 · -10^4 ≤ nums[i] ≤ 10^4

Brute-Force Approach

Try every subsequence and check if increasing: O(2^n).

Optimal Approach

DP: dp[i] = length of the longest increasing subsequence ENDING at index i = 1 + max(dp[j] for all j < i where nums[j] < nums[i]). Answer is max(dp). Optimize to O(n log n) with a 'tails' array + binary search, where tails[k] = smallest possible tail value of an increasing subsequence of length k+1.

Step-by-Step Intuition

The O(n²) DP asks 'what's the best subsequence ending here'. The faster approach instead greedily maintains, for each possible LENGTH, the smallest tail value achievable — smaller tails leave more room for future extension, and binary search finds where a new number fits in O(log n).

Dry Run

`nums=[10,9,2,5,3,7,101,18]. tails=[]. 10→tails=[10]. 9<10→replace→tails=[9]. 2<9→replace→tails=[2]. 5>2→append→tails=[2,5]. 3 fits between→replace5→tails=[2,3]. 7>3→append→tails=[2,3,7]. 101>7→append→tails=[2,3,7,101]. 18 fits before101→replace→tails=[2,3,7,18]. Length of tails=4 → answer=4.`

Time Complexity

O(n²) for the straightforward DP, O(n log n) with the tails + binary search optimization.

Space Complexity

O(n) for the DP array or the tails array.

Common Mistakes

Believing the 'tails' array IS a valid actual subsequence (it's not — it's only correct for tracking the LENGTH, not reconstructing the sequence directly without extra bookkeeping); confusing strictly increasing with non-decreasing.

Interview Follow-up Questions

- How would you reconstruct the actual longest increasing subsequence, not just its length?
- How would you find the longest COMMON increasing subsequence between two arrays?

Clean Python Solution

```
import bisect
def length_of_lis(nums):
    tails = []
    for num in nums:
        pos = bisect.bisect_left(tails, num)
        if pos == len(tails):
            tails.append(num)
        else:
            tails[pos] = num
    return len(tails)
```

KEY TAKEAWAY

When O(n²) DP is too slow, look for a greedy invariant (smallest tail per length) that binary search can maintain in O(log n).

QUESTION 66 OF 75

Medium

Coin Change

Pattern: 1D DP (unbounded knapsack)

WHY THIS QUESTION MATTERS

The canonical unbounded-knapsack DP — minimum count with unlimited reuse of each item.

Problem Statement

Given coins of different denominations and a total amount, return the fewest number of coins needed to make up that amount. Return -1 if it's not possible.

Example

```
Input: coins = [1,2,5], amount = 11 →
Output: 3  (5+5+1)
```

Constraints

1 ≤ coins.length ≤ 12 · 1 ≤ coins[i] ≤ 2^31-1 · 0 ≤ amount ≤ 10^4

Brute-Force Approach

Try every combination of coins recursively without memoization: exponential.

Optimal Approach

Bottom-up DP: dp[a] = minimum coins to make amount a. dp[0] = 0; for every amount from 1 to target, dp[a] = min(dp[a - coin] + 1) over all coins that fit.

Step-by-Step Intuition

The minimum coins to make amount a is 1 (for whichever coin you pick last) plus the minimum coins to make the remainder (a - that coin) — try every coin as 'the last one used' and take the best.

Dry Run

```
coins=[1,2,5], amount=11. dp[0]=0. dp[1]=dp[0]+1=1.
dp[2]=min(dp[1]+1,dp[0]+1)=1. dp[3]=min(dp[2]+1,dp[1]+1)=2. ...
dp[11]=min(dp[10]+1,dp[9]+1,dp[6]+1). Working through the table, dp[11]=3
(5+5+1).
```

Time Complexity

O(amount · coins.length).

Space Complexity

O(amount) for the DP array.

Common Mistakes

Using a greedy 'always pick the largest coin' approach (fails for non-canonical coin systems, e.g. coins=[1,3,4], amount=6 — greedy gives 4+1+1=3 coins but optimal is 3+3=2); forgetting to initialize unreachable amounts to infinity (not 0).

Interview Follow-up Questions

• How would you count the NUMBER OF WAYS to make the amount, instead of the minimum coins (Coin Change II)?

• How would you also return which coins were used, not just the count?

Clean Python Solution

```python
def coin_change(coins, amount):
    dp = [float('inf')] * (amount + 1)
    dp[0] = 0
    for a in range(1, amount + 1):
        for coin in coins:
            if coin <= a:
                dp[a] = min(dp[a], dp[a - coin] + 1)
    return dp[amount] if dp[amount] != float('inf') else -1
```

KEY TAKEAWAY

Unbounded knapsack DP tries every item as 'the last one used' at each target value, reusing the same DP table for all quantities.

QUESTION 67 OF 75

Medium

Unique Paths

Pattern: 2D DP (grid path counting)

WHY THIS QUESTION MATTERS

The simplest 2D DP problem — introduces building a DP table across two dimensions.

Problem Statement

A robot is at the top-left of an m x n grid and can only move right or down. Return the number of unique paths to reach the bottom-right corner.

Example

`Input: m = 3, n = 7 → Output: 28`

Constraints

1 ≤ m, n ≤ 100

Brute-Force Approach

Recursively try both moves (right, down) from every cell without memoization: O(2^(m+n)).

Optimal Approach

DP table where dp[r][c] = dp[r-1][c] + dp[r][c-1] (paths from above plus paths from the left); first row and first column are all 1 (only one way to reach them, moving straight).

Step-by-Step Intuition

To reach any cell, the robot's last move was either FROM ABOVE or FROM THE LEFT — so the number of ways to reach a cell is the sum of ways to reach those two neighbors.

Dry Run

`m=3,n=3 (smaller example). dp = [[1,1,1],[1,2,3],[1,3,6]]. Row0 and col0 are all 1. dp[1][1]=dp[0][1]+dp[1][0]=1+1=2. dp[2][2]=dp[1][2]+dp[2][1]=3+3=6. Bottom-right=6 unique paths.`

Time Complexity

`O(m·n).`

Space Complexity

`O(n) using a single rolling row instead of the full O(m·n) table.`

Common Mistakes

Forgetting to initialize the first row/column to 1 (not 0); recomputing with plain recursion, causing exponential blowup without memoization.

Interview Follow-up Questions

- How would you solve this if some cells were BLOCKED obstacles (Unique Paths II)?
- Can you derive a closed-form combinatorial formula instead of DP (hint: it's 'm+n-2 choose m-1')?

Clean Python Solution

```
def unique_paths(m, n):
    dp = [1] * n
    for _ in range(1, m):
        for c in range(1, n):
            dp[c] += dp[c - 1]
    return dp[-1]
```

KEY TAKEAWAY

Grid path-counting DP sums contributions from 'above' and 'left' — a direct 2D extension of 1D DP.

QUESTION 68 OF 75

Medium

Longest Common Subsequence

Pattern: 2D DP (string alignment)

WHY THIS QUESTION MATTERS

The foundational 2D string DP — the base of diff tools, edit distance, and many string-alignment problems.

Problem Statement

Given two strings text1 and text2, return the length of their longest common subsequence (not necessarily contiguous, but in order). Return 0 if none exists.

Example

`Input: text1 = "abcde", text2 = "ace"`
`→ Output: 3  ("ace")`

Constraints

1 ≤ text1.length, text2.length ≤ 1000

Brute-Force Approach

Try every subsequence of text1 and check if it's a subsequence of text2: exponential.

Optimal Approach

2D DP table dp[i][j] = LCS length of text1[:i] and text2[:j]. If the characters match, dp[i][j] = 1 + dp[i-1][j-1]; otherwise dp[i][j] = max(dp[i-1][j], dp[i][j-1]).

Step-by-Step Intuition

At each pair of positions, either the characters match (extend the LCS found so far by 1) or they don't (the LCS must come from ignoring one character from either string) — take whichever choice is better.

Dry Run

`text1="abcde", text2="ace". Building the table: 'a'=='a'→dp grows by 1. 'b' vs 'c' mismatch→take max of neighbors. 'c'=='c'→extend. 'd' vs 'e' mismatch. 'e'=='e'→extend. Final dp[5][3]=3 → LCS length 3 ("ace").`

Time Complexity

O(m·n).

Space Complexity

O(m·n), reducible to O(min(m,n)) with a rolling 2-row table.

Common Mistakes

Confusing subsequence (can skip characters, order preserved) with substring (must be contiguous) — a very common misunderstanding; off-by-one indexing between the DP table (1-indexed conceptually) and the strings (0-indexed).

Interview Follow-up Questions

• How would you reconstruct the actual LCS string, not just its length?

• How does this relate to Edit Distance (Levenshtein), and can you adapt the DP recurrence?

Clean Python Solution

```
def longest_common_subsequence(text1, text2):
    m, n = len(text1), len(text2)
    dp = [[0] * (n + 1) for _ in range(m + 1)]
    for i in range(1, m + 1):
        for j in range(1, n + 1):
            if text1[i - 1] == text2[j - 1]:
                dp[i][j] = 1 + dp[i - 1][j - 1]
            else:
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1])
    return dp[m][n]
```

KEY TAKEAWAY

2D string-alignment DP compares two sequences character by character, branching on match vs. no-match at every cell.

QUESTION 69 OF 75

Medium

Word Break

Pattern: 1D DP + Hashing (segmentation)

WHY THIS QUESTION MATTERS

Shows DP applied to a segmentation problem — 'can this be split into valid pieces' — a different DP shape than counting/optimizing.

Problem Statement

Given a string s and a dictionary of strings wordDict, return true if s can be segmented into a space-separated sequence of one or more dictionary words.

Example

`Input: s = "leetcode", wordDict = ["leet","code"] → Output: true`

Constraints

1 ≤ s.length ≤ 300 · 1 ≤ wordDict.length ≤ 1000 · words consist of lowercase letters

Brute-Force Approach

Try every possible split point recursively without memoization: exponential, O(2^n).

Optimal Approach

DP where dp[i] = true if s[:i] can be segmented using dictionary words. dp[0] = true (empty prefix). For each i, check every j < i: if dp[j] is true AND s[j:i] is in the word set, set dp[i] = true.

Step-by-Step Intuition

A prefix of length i is breakable if there's SOME earlier breakable prefix j such that the remaining chunk s[j:i] is itself a valid word — build up from the empty prefix and let each valid split point unlock further prefixes.

> **Dry Run**
>
> `s="leetcode", dict={"leet","code"}. dp[0]=True. Check i=4: j=0, s[0:4]="leet" in dict and dp[0]=True → dp[4]=True. Check i=8: j=4, s[4:8]="code" in dict and dp[4]=True → dp[8]=True. dp[8] (full string length) is True → answer True.`

Time Complexity

$O(n^2)$ or $O(n^2 \cdot k)$ depending on substring-slicing cost, where n = len(s).

Space Complexity

O(n) for the DP array plus O(sum of word lengths) for the word set.

Common Mistakes

Forgetting dp[0] = True as the base case (empty prefix is trivially breakable); recomputing substring membership without a set (checking against a list is O(k) per check instead of O(1)).

Interview Follow-up Questions

- How would you return ALL possible valid segmentations, not just true/false (Word Break II)?
- How would a Trie speed up substring matching for a very large dictionary?

Clean Python Solution

```
def word_break(s, word_dict):
    words = set(word_dict)
    n = len(s)
    dp = [False] * (n + 1)
    dp[0] = True
    for i in range(1, n + 1):
        for j in range(i):
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break
    return dp[n]
```

KEY TAKEAWAY

Segmentation DP marks which PREFIXES are achievable, extending from each achievable prefix by testing valid next chunks.

CATEGORY 13 OF 14

Greedy

Make the locally optimal choice at each step and prove it leads to a globally optimal answer.

4 questions in this section

QUESTION 70 OF 75

Medium

Jump Game

Pattern: Greedy (reachability frontier)

WHY THIS QUESTION MATTERS

The introductory greedy-reachability problem — track the furthest index reachable so far instead of exploring every path.

Problem Statement

Given an array nums where nums[i] is the maximum jump length from index i, return true if you can reach the last index starting from index 0.

Example

```
Input: nums = [2,3,1,1,4] → Output: true
Input: nums = [3,2,1,0,4] → Output: false
```

Constraints

1 ≤ nums.length ≤ 10^4 · 0 ≤ nums[i] ≤ 10^5

Brute-Force Approach

Try every combination of jumps recursively (backtracking through all reachable indices): exponential.

Optimal Approach

Greedily track the farthest index reachable so far while scanning left to right. If the current index ever exceeds that farthest reach, it's unreachable — return false. Otherwise update farthest = max(farthest, i + nums[i]).

Step-by-Step Intuition

You never need to know WHICH path gets you furthest, only the single best (furthest) reach achievable by any index processed so far — a single greedy scan captures that.

Dry Run

```
nums=[2,3,1,1,4]. farthest=0. i=0: 0<=farthest(0) ok, farthest=max(0,0+2)=2.
i=1: 1<=2 ok, farthest=max(2,1+3)=4. i=2: 2<=4 ok, farthest=max(4,2+1)=4. i=3:
farthest=max(4,3+1)=4. i=4(last index): 4<=4 ok → reachable → True.
```

Time Complexity

Space Complexity

O(n) — single pass.

O(1).

Common Mistakes

Using DP (O(n²)) when the greedy O(n) approach is expected and simpler; forgetting to check reachability of the CURRENT index before updating farthest (an unreachable gap must fail immediately).

Interview Follow-up Questions

• How would you find the MINIMUM number of jumps to reach the end (Jump Game II)?

• What if you needed to reach a specific index, not just the last one?

Clean Python Solution

```
def can_jump(nums):
    farthest = 0
    for i, num in enumerate(nums):
        if i > farthest:
            return False
        farthest = max(farthest, i + num)
    return True
```

KEY TAKEAWAY

When only the BEST reach matters (not the path), track a single running frontier greedily instead of exploring every option.

QUESTION 71 OF 75

Medium

Gas Station

Pattern: Greedy (running total reset)

WHY THIS QUESTION MATTERS

A clever greedy proof question — shows how a single running total identifies both feasibility and the starting point.

Problem Statement

There are n gas stations in a circle, each with gas[i] fuel and cost[i] to travel to the next station. Return the starting station index if you can complete the circuit exactly once, or -1 if impossible (the answer is guaranteed unique if it exists).

Example

`Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2] → Output: 3`

Constraints

n == gas.length == cost.length · 1 ≤ n ≤ 10^5 · 0 ≤ gas[i], cost[i] ≤ 10^4

Brute-Force Approach

Try starting from every station and simulate the full circuit: O(n²).

Optimal Approach

Track a running tank total while scanning once. If the tank ever goes negative at station i, no start from the previous candidate through i can work — reset the candidate start to i+1 and reset the tank to 0. Separately verify total gas ≥ total cost overall (necessary for any solution to exist).

Step-by-Step Intuition

If the tank goes negative arriving at some station, EVERY station between the current candidate start and that failure point is also a doomed starting point (they'd fail even sooner or at the same spot) — so you can safely skip straight past all of them.

Dry Run

`gas=[1,2,3,4,5], cost=[3,4,5,1,2]. total=0,tank=0,start=0. i=0: diff=1-3=-2, total=-2,tank=-2<0→start=1,tank=0. i=1: diff=2-4=-2,total=-4,tank=-2<0→start=2,tank=0. i=2: diff=3-5=-2,total=-6,tank=-2<0→start=3,tank=0. i=3: diff=4-1=3,total=-3,tank=3. i=4: diff=5-2=3,total=0,tank=6. Final total=0 (>=0, feasible) → answer=start=3.`

Time Complexity

`O(n) — single pass.`

Space Complexity

`O(1).`

Common Mistakes

Using the O(n²) brute-force simulation when O(n) greedy is expected; forgetting to check total gas ≥ total cost overall (a candidate start could be found even when no solution exists, without this check).

Interview Follow-up Questions

- Can you prove why resetting the candidate start after a negative tank is always safe?
- How would this change if you could complete the circuit going in either direction?

Clean Python Solution

```
def can_complete_circuit(gas, cost):
    total, tank, start = 0, 0, 0
    for i in range(len(gas)):
        diff = gas[i] - cost[i]
        total += diff
        tank += diff
        if tank < 0:
            start = i + 1
            tank = 0
    return start if total >= 0 else -1
```

KEY TAKEAWAY

When a running total goes negative, everything since the last reset point is provably a bad start — skip past all of it at once.

QUESTION 72 OF 75

Medium

Non-overlapping Intervals

Pattern: Greedy (interval scheduling by end time)

WHY THIS QUESTION MATTERS

The classic interval-scheduling greedy — sort by end time, a technique that reappears constantly.

Problem Statement

Given an array of intervals, return the minimum number of intervals you need to remove to make the rest non-overlapping.

Example

```
Input: intervals = [[1,2],[2,3],[3,4],
[1,3]] → Output: 1  (remove [1,3])
```

Constraints

1 ≤ intervals.length ≤ 10^5 · intervals[i].length == 2

Brute-Force Approach

Try every subset of intervals to keep and check for non-overlap: exponential.

Optimal Approach

Sort intervals by END time. Greedily keep an interval if it starts at or after the end of the last kept interval; otherwise it must be removed (increment a removal counter).

Step-by-Step Intuition

Among overlapping intervals, keeping the one that ends EARLIEST leaves the most room for future intervals to also fit without overlapping — this is the provably optimal greedy choice (an exchange argument).

> **Dry Run**
>
> `Sorted by end: [1,2],[2,3],[1,3],[3,4]. lastEnd=-inf. [1,2]: 1>=lastEnd(-inf)→keep,lastEnd=2. [2,3]: 2>=2→keep,lastEnd=3. [1,3]: 1<3→overlap→remove,count=1. [3,4]: 3>=3→keep,lastEnd=4. Total removed=1.`

Time Complexity

`O(n log n) — dominated by the sort.`

Space Complexity

`O(1) extra beyond sorting.`

Common Mistakes

Sorting by START time instead of END time (the greedy proof only holds for sorting by end time); off-by-one on the overlap condition (should be strictly less-than for the start vs lastEnd comparison).

Interview Follow-up Questions

- How would you solve 'Merge Intervals' with the same sort-by-start-time family of techniques?
- How would you find the MAXIMUM number of non-overlapping intervals directly (equivalent, but framed positively)?

Clean Python Solution

```python
def erase_overlap_intervals(intervals):
    intervals.sort(key=lambda x: x[1])
    count = 0
    last_end = float('-inf')
    for start, end in intervals:
        if start >= last_end:
            last_end = end
        else:
            count += 1
    return count
```

KEY TAKEAWAY

Interval scheduling for maximum non-overlap always sorts by END time — keep the interval that frees up room soonest.

QUESTION 73 OF 75

Medium

Merge Intervals

Pattern: Greedy + Sorting (interval merge)

WHY THIS QUESTION MATTERS

Complements interval scheduling with the merge variant, sorted by start time — both orderings are essential to know.

Problem Statement

Given an array of intervals, merge all overlapping intervals and return the resulting non-overlapping intervals covering all the input.

Example

```
Input: intervals = [[1,3],[2,6],
[8,10],[15,18]] → Output: [[1,6],
[8,10],[15,18]]
```

Constraints

1 ≤ intervals.length ≤ 10^4 · intervals[i].length == 2

Brute-Force Approach

Repeatedly scan all pairs, merging any that overlap, until no more merges are possible: O(n²) or worse.

Optimal Approach

Sort intervals by START time. Walk through, keeping a 'current merged interval'; if the next interval's start is ≤ the current merged interval's end, extend the end (max of both ends); otherwise, close off the current merged interval and start a new one.

Step-by-Step Intuition

Once sorted by start time, any interval that overlaps the current merge candidate MUST appear immediately next (nothing between them in sorted order could overlap without also overlapping the current one) — a single linear scan suffices.

Dry Run

```
Sorted: [1,3],[2,6],[8,10],[15,18]. current=[1,3]. Next[2,6]:
2<=3→merge→current=[1,6]. Next[8,10]: 8>6→close current, output[1,6], current=
[8,10]. Next[15,18]: 15>10→close, output[8,10], current=[15,18].
End→output[15,18]. Result=[[1,6],[8,10],[15,18]].
```

Time Complexity

O(n log n) — dominated by the sort.

Space Complexity

O(n) for the output (O(log n) to O(n) for the sort itself, depending on implementation).

Common Mistakes

Sorting by end time instead of start time (breaks the linear-merge assumption); comparing with strict < instead of ≤ for the overlap check (misses intervals that exactly touch, like [1,3] and [3,5], if the problem considers touching as overlapping).

Interview Follow-up Questions

• How would you INSERT a new interval into an already-sorted, already-merged list efficiently (Insert Interval)?

• How would you find the minimum number of meeting rooms needed for a set of intervals (a related but distinct problem)?

Clean Python Solution

```
def merge_intervals(intervals):
    intervals.sort(key=lambda x: x[0])
    merged = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])
    return merged
```

KEY TAKEAWAY

Sorting by START time turns interval merging into a single linear scan — overlaps can only occur with the immediately preceding interval.

CATEGORY 14 OF 14

Bit Manipulation

XOR and bit tricks that solve problems in O(1) space that would otherwise need extra memory.

2 questions in this section

QUESTION 74 OF 75

Easy

Single Number

Pattern: Bit Manipulation (XOR)

WHY THIS QUESTION MATTERS

The essential XOR trick — a self-cancelling operation that solves an O(n) space problem in O(1) space.

Problem Statement

Given a non-empty array of integers nums where every element appears exactly twice except for one, find that single one, in O(n) time and O(1) space.

Example

`Input: nums = [4,1,2,1,2] → Output: 4`

Constraints

1 ≤ nums.length ≤ 3×10^4 · each element appears twice except one, which appears once

Brute-Force Approach

Use a hash map to count occurrences, then find the one with count 1: O(n) time, O(n) space.

Optimal Approach

XOR every element together. Since a XOR a = 0 for any a, and XOR is commutative/associative, all paired elements cancel out, leaving only the single unpaired element.

Step-by-Step Intuition

XOR is its own inverse (x^x=0) and order-independent, so XOR-ing the entire array collapses every duplicate pair to zero, and zero XOR anything returns that thing unchanged — leaving exactly the unpaired value.

Dry Run

`nums=[4,1,2,1,2]. result=0. 0^4=4. 4^1=5. 5^2=7. 7^1=6 (since 5^2^1... let's just trust associativity: 4^1^2^1^2 = 4^(1^1)^(2^2) = 4^0^0 = 4). Answer=4.`

Time Complexity

`O(n) — single pass.`

Space Complexity

`O(1) — no hash map needed, unlike the brute force.`

Common Mistakes

Using a hash map/set when O(1) space is explicitly required; forgetting XOR is commutative (so the order elements appear in the array doesn't matter for the cancellation).

Interview Follow-up Questions

• How would you solve this if every element appeared THREE times except one (Single Number II)?

• How would you find TWO single numbers if exactly two elements are unpaired (Single Number III)?

Clean Python Solution

```
def single_number(nums):
    result = 0
    for num in nums:
        result ^= num
    return result
```

KEY TAKEAWAY

XOR cancels identical pairs to zero — whenever 'everything appears twice except one', XOR the whole array.

QUESTION 75 OF 75

Easy

Number of 1 Bits

Pattern: Bit Manipulation (bit clearing trick)

WHY THIS QUESTION MATTERS

A simple but interview-common bit trick: n & (n-1) clears the lowest set bit, giving an elegant O(popcount) loop.

Problem Statement

Write a function that takes an unsigned integer and returns the number of '1' bits it has (its Hamming weight).

Example

`Input: n = 11 (binary 1011) → Output: 3`

Constraints

the input is a 32-bit unsigned integer

Brute-Force Approach

Check each of the 32 bits individually with a mask and shift: O(32), constant but does unnecessary work for sparse numbers.

Optimal Approach

Repeatedly apply n = n & (n - 1), which clears the lowest set bit each time; count how many iterations it takes until n becomes 0.

Step-by-Step Intuition

Subtracting 1 from n flips all trailing zero bits to 1 and the lowest set bit to 0; ANDing with the original n then clears exactly that lowest set bit and nothing else — so each iteration removes exactly one '1' bit.

> **Dry Run**

n=11 (1011). Iteration1: n&(n-1) = 1011 & 1010 = 1010 (10), count=1. Iteration2: 1010 & 1001 = 1000 (8), count=2. Iteration3: 1000 & 0111 = 0000, count=3. n=0→stop. Answer=3.

Time Complexity

O(k) where k = number of set bits (at most 32), faster than checking all 32 bits when the number is sparse.

Space Complexity

O(1).

Common Mistakes

Checking all 32 bit positions unconditionally when a tighter loop (bounded by set-bit count) is expected as a follow-up; not handling the 'unsigned' nature correctly in languages with signed integer quirks (less of an issue in Python).

Interview Follow-up Questions

• How would you precompute the Hamming weight for every number from 0 to n efficiently (Counting Bits)?

• Can you do this with a built-in population-count instruction/function, and when would you prefer the manual version?

Clean Python Solution

```
def hamming_weight(n):
    count = 0
    while n:
        n &= (n - 1)
        count += 1
    return count
```

KEY TAKEAWAY

n & (n-1) clears the lowest set bit — a fast, elegant way to count or manipulate set bits without checking every position.

SECTION 7

Final Revision Cheat Sheet

CAN YOU SOLVE THESE WITHOUT HELP?

Close the book. Give yourself 25 minutes each for: Q7 (Subarray Sum Equals K), Q11 (Container With Most Water), Q16 (Minimum Window Substring), Q21 (Koko Eating Bananas), Q34 (Reorder List), Q39 (Largest Rectangle in Histogram), Q50 (Find Median from Data Stream), Q55 (N-Queens), Q60 (Pacific Atlantic Water Flow), Q68 (Longest Common Subsequence).

If you can solve 8 of 10 cleanly — without peeking — you are interview-ready.

Top 15 Questions to Revise Before an Interview

- Q1 · Two Sum
- Q7 · Subarray Sum Equals K
- Q10 · 3Sum
- Q14 · Longest Substring Without Repeating Characters
- Q19 · Search in Rotated Sorted Array
- Q29 · Reverse Linked List
- Q33 · Remove Nth Node From End of List
- Q73 · Merge Intervals
- Q40 · Invert Binary Tree
- Q44 · Validate Binary Search Tree
- Q47 · Kth Largest Element in an Array
- Q51 · Subsets
- Q56 · Number of Islands
- Q58 · Course Schedule
- Q64 · House Robber

Top DSA Patterns to Memorize

Two Pointers · Sliding Window · Prefix Sum · Fast & Slow Pointers · Binary Search on Answer · Monotonic Stack · BFS · DFS · Topological Sort · Heap / Priority Queue · Backtracking · Greedy · 1D DP · 2D DP

Last-Minute Complexity Cheat Sheet

Hash map ops: O(1) avg · Sorting: O(n log n) · Binary search: O(log n) · Heap push/pop: O(log n) · Tree traversal: O(n) · Graph traversal: O(V+E) · DP table fill: O(states × transitions)

SECTION 8

Interview-Day Checklist

Before You Start Coding

☐ Repeat the problem back in your own words

☐ Confirm input size, constraints, and edge cases

☐ State the brute-force approach and its complexity out loud

☐ Name the pattern you recognize before coding

☐ Confirm the optimal approach with the interviewer before writing code

While Coding

☐ Think out loud, don't code in silence

☐ Use meaningful variable names, not single letters

☐ Handle empty input / null / single-element edge cases

☐ Dry-run your own code on the example before saying "done"

☐ State final time and space complexity unprompted

Common Red Flags to Avoid

☐ Jumping to code before agreeing on approach

☐ Ignoring the interviewer's hints

☐ Silence for more than 30 seconds

☐ Leaving obvious bugs unaddressed after a hint

Night Before

☐ Skim the Pattern Cheat Sheet, not new problems

☐ Re-read your Top 15 revision list

☐ Sleep — a rested brain outperforms one more practice problem

SECTION 9

Progress Tracker

Check off each question as you complete it without help.

- ☐ Q1 · Two Sum
- ☐ Q2 · Contains Duplicate
- ☐ Q3 · Valid Anagram
- ☐ Q4 · Group Anagrams
- ☐ Q5 · Top K Frequent Elements
- ☐ Q6 · Product of Array Except Self
- ☐ Q7 · Subarray Sum Equals K
- ☐ Q8 · Valid Palindrome
- ☐ Q9 · Two Sum II — Sorted Array
- ☐ Q10 · 3Sum
- ☐ Q11 · Container With Most Water
- ☐ Q12 · Trapping Rain Water
- ☐ Q13 · Best Time to Buy and Sell Stock
- ☐ Q14 · Longest Substring Without Repeating Characters
- ☐ Q15 · Longest Repeating Character Replacement
- ☐ Q16 · Minimum Window Substring
- ☐ Q17 · Sliding Window Maximum
- ☐ Q18 · Binary Search
- ☐ Q19 · Search in Rotated Sorted Array
- ☐ Q20 · Find Minimum in Rotated Sorted Array
- ☐ Q21 · Koko Eating Bananas
- ☐ Q22 · Find First and Last Position of Element in Sorted Array
- ☐ Q23 · Median of Two Sorted Arrays
- ☐ Q24 · Longest Common Prefix
- ☐ Q25 · Longest Palindromic Substring
- ☐ Q26 · Palindromic Substrings
- ☐ Q27 · String to Integer (atoi)
- ☐ Q28 · Encode and Decode Strings
- ☐ Q29 · Reverse Linked List
- ☐ Q30 · Linked List Cycle
- ☐ Q31 · Middle of the Linked List
- ☐ Q32 · Merge Two Sorted Lists
- ☐ Q33 · Remove Nth Node From End of List
- ☐ Q34 · Reorder List
- ☐ Q35 · Valid Parentheses
- ☐ Q36 · Min Stack
- ☐ Q37 · Evaluate Reverse Polish Notation
- ☐ Q38 · Daily Temperatures
- ☐ Q39 · Largest Rectangle in Histogram
- ☐ Q40 · Invert Binary Tree
- ☐ Q41 · Maximum Depth of Binary Tree
- ☐ Q42 · Diameter of Binary Tree
- ☐ Q43 · Binary Tree Level Order Traversal
- ☐ Q44 · Validate Binary Search Tree
- ☐ Q45 · Lowest Common Ancestor of a BST
- ☐ Q46 · Kth Smallest Element in a BST
- ☐ Q47 · Kth Largest Element in an Array
- ☐ Q48 · K Closest Points to Origin
- ☐ Q49 · Merge K Sorted Lists
- ☐ Q50 · Find Median from Data Stream
- ☐ Q51 · Subsets
- ☐ Q52 · Permutations
- ☐ Q53 · Combination Sum
- ☐ Q54 · Word Search
- ☐ Q55 · N-Queens
- ☐ Q56 · Number of Islands
- ☐ Q57 · Clone Graph
- ☐ Q58 · Course Schedule
- ☐ Q59 · Course Schedule II
- ☐ Q60 · Pacific Atlantic Water Flow
- ☐ Q61 · Graph Valid Tree
- ☐ Q62 · Network Delay Time
- ☐ Q63 · Climbing Stairs
- ☐ Q64 · House Robber
- ☐ Q65 · Longest Increasing Subsequence
- ☐ Q66 · Coin Change
- ☐ Q67 · Unique Paths
- ☐ Q68 · Longest Common Subsequence
- ☐ Q69 · Word Break

☐ Q70 · Jump Game

☐ Q71 · Gas Station

☐ Q72 · Non-overlapping Intervals

☐ Q73 · Merge Intervals

☐ Q74 · Single Number

☐ Q75 · Number of 1 Bits

Completed all 75? You now recognize every major interview pattern on sight. Return to the Final Revision Cheat Sheet and re-solve the Top 15 cold, without notes, one more time before your interview.